

基于非线性规划的蒙特卡洛模拟种植方案求解

摘要

乡村农作物种植策略往往具有多种选择。针对不同类型的耕地资源，需要因地制宜，选择适宜生长的作物。不同作物的经济效益以及每季的预计需求量也指导着不同作物在量上的耕种选择。除此之外，种植方案的选择还要考虑多种因素，包括耕种与管理的便利性。优化种植方案，充分利用不同类型耕地资源，对乡村经济发展有着重要意义。

对于问题一，考虑到耕种资源管理的便利性，我们将不同耕地的面积离散化，按其总面积分成数等份，每份土地进行统一耕种。参考 2023 年的种植情况，可将大棚耕地分为两份，而非大棚地块不进行划分，作为单独一份。除了豆类作物的种植需要考量，各年的种植策略相对独立，可依次求解。因此我们以某作物在某季某耕地上种植所占的份数为决策变量，农作物总利润作为优化目标，而其他条件因素作为约束条件，可构建**非线性规划模型**。对于模型复杂性太高的问题，可以使用**经约束条件优化的蒙特卡洛模拟算法**在大量可行解中选出最优方案。不同销售情况（滞销与降价）通过更换优化目标进行模拟。最终结果得到 2024~2030 七年的利润总和在滞销销售情况下约为 **29610565 元**（约三千万），而在降价销售情况下约为 **49422364 元**（约五千万）。

对于问题二，需要在问题一的基础上考虑农作物未来每年销售数据、成本与产量的变动。对于变动范围较大的数值，我们引入**截断正态分布**进行随机变动率的生成，从而使结果更符合生产销售实际。通过随机变动率改变原来的预期数值，再次使用问题一中的模型与算法进行求解（仅考虑滞销情况），最终结果得到 2024~2030 七年的利润总和约为 **30842801 元**（约三千万）。

对于问题三，需要在问题二的基础上考虑数据间的相关性。对于作物间的可替代性，我们按作物类型将农作物划分为五个组别。假设同一组别中作物之间存在可替代性，其中某一农作物价格上涨便会造成其他作物预期销售量的上升。对于作物间的互补性，我们从耕种产量角度进行分析，即豆类作物的耕种可提高其他作物的产量。对于经济数据间的相关性，我们引入**灰色关联分析**，以预期销售量为母序列，销售价格与种植成本为子序列计算灰色关联度，最后通过关联度定义子序列对应变量对母序列对应变量的作用公式。综合上述因素，我们将经过调整的数据用问题一中的模型与算法进行求解（仅考虑滞销情况），最终得到未来七年利润总和约为 **25922292 元**（约两千六百万），与问题二的结果相比较低。

最后，我们对蒙特卡洛模拟算法进行了合理性分析，并对问题三的模型进行了敏感性分析，以提高模型的合理性和准确性。

关键字： 非线性规划 蒙特卡洛模拟 截断正态分布 灰色关联分析

一、问题重述

1.1 问题背景

对于乡村种植策略的选择，需要考虑到提高生产效益，规避风险，便于管理等因素。根据题目所给信息，各因素的具体形式可概括为：

- **提高生产效益：**所有土地三年内至少种植一次豆类作物，从而丰富土壤中豆类作物根瘤菌的含量。各季土地选择适宜生长的农作物进行耕种，且需根据预期销售量进行调整，从而最大化农作物利润。
- **规避风险：**避免因连续重茬种植而造成不必要的减产。例如玉米重茬种植会加重作物病害与虫害的情况，且导致作物所需养分严重失衡 [1]。
- **便于管理：**为了方便耕种及管理的进行，减轻农民负担，每季中各种类农作物尽量集中在少数耕地（即地块和大棚）中，且在单个耕地中种植面积不宜过小。

现有华北山区某乡村的耕作信息，乡村共有 6 个平旱地、14 个梯田、6 个山坡地、8 个水浇地、16 个普通大棚及 4 个智慧大棚，其中大棚的耕地面积都为 0.6 亩，其他类型耕地面积大小不一。各类型耕地在不同季度适宜种植的农作物范围不一，其中粮食作物只能单季种植。附件 1 分别提供了各耕地信息（表单 1）及各农作物信息（表单 2）；附件 2 中，表单 1 给出了 2023 年的种植情况，表单 2 提供了 2023 年各季各作物的产量、种植成本及销售单价范围。

1.2 问题重述

问题一：根据两种销售方案分别构建该乡村 2024~2030 年农作物种植方案的优化模型，其中各农作物的预期销售量、种植成本、亩产量及销售单价都参照 2023 年的数据，且农作物都在种植季当季卖出。第一种销售方案将超出预期销售量的产量部分进行作废处理，而第二种销售方案将超出部分进行 50% 的降价出售。

问题二：在问题一模型的基础上，考虑到各农作物种植成本、销售单价、亩产量和预期销售量的变动，以及各种潜在风险因素，重新构建该乡村 2024~2030 年农作物种植方案的优化模型。题目所提供的变动条件可归纳如下：

- **预期销售量：**小麦和玉米预期销售量平均年增长率在 5%~10% 之间；其他作物每年预期销售量在 2023 年数据基础上有 $\pm 5\%$ 的波动。
- **亩产量：**每年相比于 2023 年有 $\pm 10\%$ 的波动（受气候等因素影响）。
- **种植成本：**平均每年增长约 $\pm 5\%$ （受市场等因素影响）。

- 销售单价：粮食类作物价格保持稳定；蔬菜类作物价格平均每年增长约 $\pm 5\%$ ；食用菌价格每年约下降 $1\% \sim 5\%$ ，其中羊肚菌价格约下降 5% 。

问题三：在问题二模型的基础上进行改进，重新构建优化模型，新增考虑因素包括：

- 农作物之间存在的可替换性和互补性。
- 预期销售量与销售单价、种植成本间的相关性。

二、 问题分析

2.1 问题 1 的分析

问题 1 要求给出该乡村 2024~2030 年的最佳耕种方案。根据题目所给条件，我们可以沿用 2023 年各农作物的预期销售量、种植成本、亩产量及销售价格对未来总利润进行求解，且农作物只在种植当季销售。由于附件未提供 2023 作物的预期销售量，我们可假定 2023 年的农作物产量即为预期销售量（考虑到该农村 2023 年的种植方案可能是依据预期销售量进行设计）。考虑到便于管理的因素，我们可将各耕地的种植面积离散化，分块进行统一的耕种（即选择种满某种作物或空缺）。

为了简化求解过程，我们依次单独对每一年的种植方案进行求解。以某农作物在某季度于某耕地耕种的面积（根据土地面积设立为离散值）为决策变量，依据各种农作物种植限制设定约束条件，并根据不同销售情况设立对应的优化目标函数，可建立非线性规划模型。考虑到模型的复杂性与解的离散性，可以使用蒙特卡洛模拟生成大量随机解。为了减小模拟的取值空间，优化求解过程，可以将部分约束条件用于限定取值范围，而其余约束条件用于筛选符合全体条件的可行解，最终在可行解中挑选出最优解作为最佳农作物种植方案。

2.2 问题 2 的分析

问题 2 需要在问题 1 的基础上引入对农作物各销售数据、成本及产量波动的考量。为了使根据波动范围生成的随机变化率更符合实际情况，我们引入截断正态分布进行随机变化率的生成。使用引入随机变动后的数据，根据问题 1 中的模型及求解算法得出最佳农作物种植方案。

2.3 问题 3 的分析

问题 3 需要在问题 2 的基础上进一步考虑不同农作物之间在销售及种植情况下的相关性，即可替代性和互补性，同时也要考虑同种农作物预期销售量与销售价格、种植成本间的相关性。由于可替代性的强弱难以统计，我们将农作物依据作物类型分为豆类粮食、非豆类粮食、豆类蔬菜、非豆类蔬菜以及食用菌这五个组别，处于同一组别的作

物具有可替代性，且某作物的价格变动会以某一比例影响其他同组别的作物。对于互补性，我们考虑到豆类与其他作物配合种植的助产效益，对相应作物进行产量的增长。对于预期销售量与销售价格、种植成本间的相关性，我们引入灰色关联分析以各作物各季的预期销售量为母序列，相应销售价格即种植成本为子序列进行关联度的计算，最后依据关联度设计两类数据对预期销售量影响的计算公式。

综合问题 2 与问题 3 中的数据变动计算方式，我们使用问题 1 的模型及求解算法得到最佳的种植方案，并与问题 2 的结果进行比较分析。

三、模型假设

1. 考虑到豆类作物根菌的促生长作用以及管理的便利性，假设当某地块（含大棚）累计两年未全面种植（即使用到所有面积）豆类作物时在下一年第一季将所有面积用于种植豆类作物。
2. 考虑到作物种植不宜太分散且在单块地上的种植面积也不宜太小，我们将大棚均分为两份土地，而非大棚地块则视为一整份土地，假设每份土地只能选择种满一种作物或不进行耕种。这种划分方法也跟附件 2 中 2023 年的种植情况基本一致，因此具有一定合理性。
3. 假设不存在某块地不论选择种植何种作物都会造成利润损失的情况，即如果有适宜种植的作物，就不会出现土地空缺的情况。
4. 假设题目中所给种植成本包含农作物种植材料费用、人工费用、工具费用等，计算利润时不包含其他额外成本。
5. 假设 2023 年各销售单价的期望值为其销售单价区间的中间值。
6. 假设 2023 年各季每种农作物的总产量等于 2023 年对应的预期销售量。考虑到 2023 年的种植方案可能是根据预期销售量制定的，因此假设具有一定合理性。由于智慧大棚第二季蔬菜数据较少，假设其预期销售量为 0.3 亩所对应产量。

四、符号说明

符号	说明	单位
d_i	地块 i 的地块面积	亩
y_i	地块 i 目前未全面种植豆类作物的累计年数	年
$m_{i,j,k,t}$	年份 t 第 k 季度地块 i 中作物 j 的种植份数	\
$x_{i,j,k,t}$	年份 t 第 k 季度地块 i 中作物 j 的种植面积	亩
$a_{i,j,k,t}$	年份 t 第 k 季度地块 i 中作物 j 的亩产量	斤/亩
$p_{i,j,k,t}$	年份 t 第 k 季度作物 j 的销售单价	元/斤
$s_{j,k,t}$	年份 t 第 k 季度作物 j 的总预期销售量	斤
$ct_{i,j,k,t}$	年份 t 第 k 季度地块 i 中作物 j 的种植成本	元/亩
I_t	年份 t 农作物总收入	元
O_t	年份 t 农作物总成本	元
W_t	年份 t 农作物总利润	元

五、数据预处理及分析

5.1 数据预处理

考虑到不同类型耕地在各季度适宜种植的农作物范围不一，我们首先根据约束条件对不同情况下可种植农作物范围进行了相应的统计。统计过程所考虑的约束条件如下：

1. 平旱地、梯田和山坡地每年只能种植单季的粮食类作物（水稻除外）。
2. 水浇地每年可以选择单季种植水稻或者两季种植蔬菜类作物，若第一季选择种植蔬菜（大白菜、白萝卜、红萝卜除外），则第二季只能种植大白菜、白萝卜、红萝卜中的一种。
3. 普通大棚第一季可种植蔬菜类作物（大白菜、白萝卜、红萝卜除外），第二季可种植食用菌。
4. 智慧大棚第一季与第二季都可种植蔬菜（大白菜、白萝卜、红萝卜除外）。

统计结果如表1所示。分析表格1数据可知，一些土地类型之间种植方案的独立性较强。如除水稻外的粮食类作物只能在平旱地、梯田及山坡地上种植，且这些土地不宜种植其他作物，因此这三类地块与其他类型土地种植方案设计相对独立性较强，而第二季当中的普通大棚与食用菌种植也属于类似情况。除此之外，水浇地的种植方案存在两种互斥的情况，即选择种植单季水稻或者两季蔬菜。

表 1 各类型耕地各季可种植的农作物

地块类型	种植季次	可种植农作物对应编号
平旱地	单季	1, 2, ... , 15
梯田	单季	1, 2, ... , 15
山坡地	单季	1, 2, ... , 15
水浇地	单季	16
水浇地	第一季	17, 18, ... , 34
水浇地	第二季	35, 36, 37 (若不种植单季水稻)
普通大棚	第一季	17, 18, ... , 34
普通大棚	第二季	38, 39, 40, 41
智慧大棚	第一季	17, 18, ... , 34
智慧大棚	第二季	17, 18, ... , 34

5.2 数据初步分析

5.2.1 农作物销售价值分析

为了对各农作物的经济效益有基本的了解，我们统计出各农作物在不同土地类型各季度种植的亩利润，即农作物在一亩土地耕种所得产量全部以期望单价卖出能得到的利润，其中期望单价取 2023 年销售价格区间的中值。通过这一指标，即可初步判断某一类型土地在各季优先种植哪些作物会得到更高的收益。统计结果如下图1、2所示（仅展示平旱地及水浇地统计结果，其余结果柱状图见附录 B）。

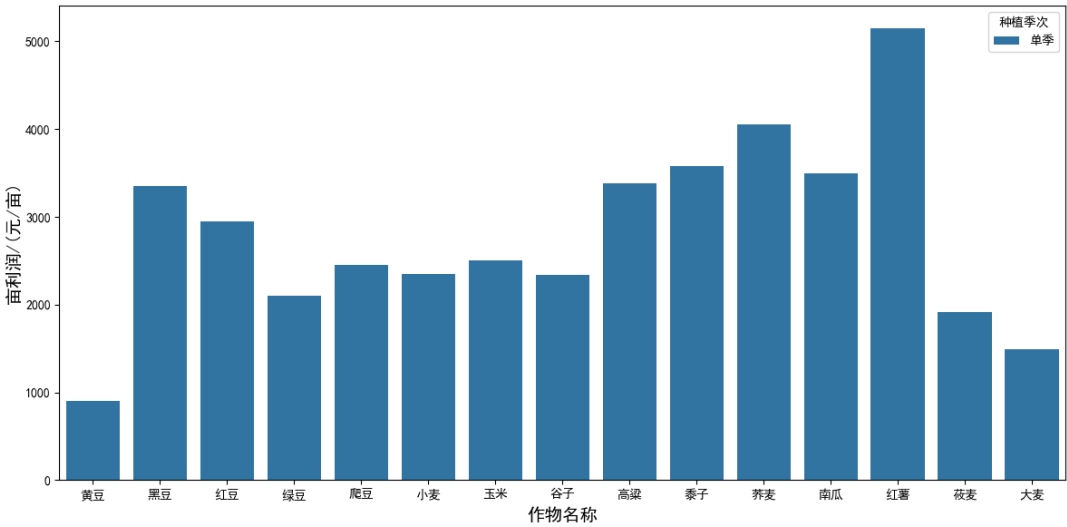


图 1 平旱地相应亩产量统计

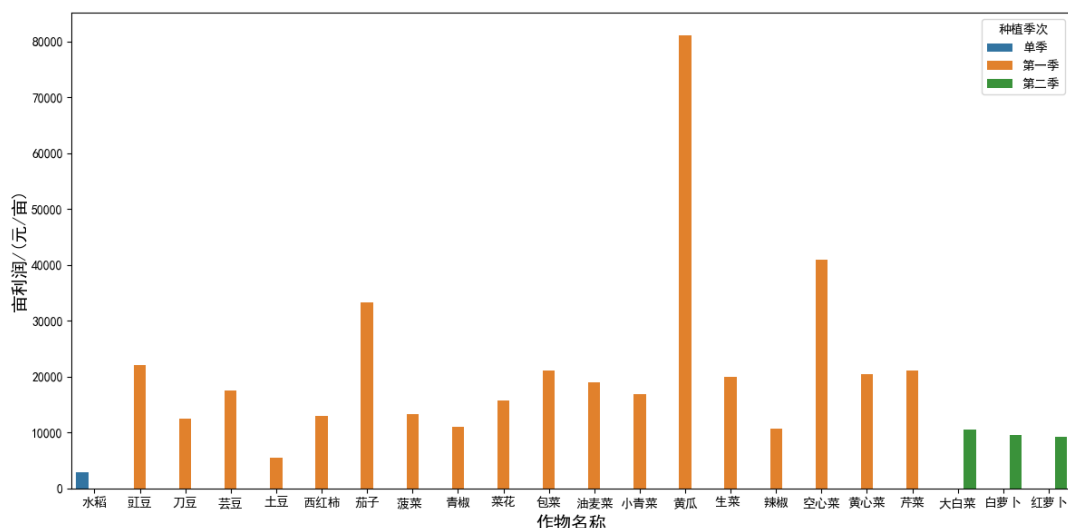


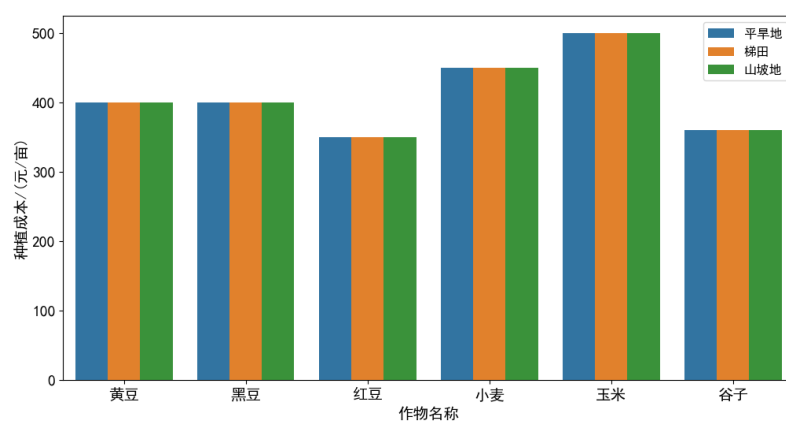
图 2 水浇地相应亩产量统计

根据柱状图，我们发现平旱地、山坡地、梯田种植的红薯，普通大棚第一季、水浇地第一季、智慧大棚第一、二季种植的黄瓜，普通大棚第二季种植的榆黄菇，水浇地第二季的大白菜有最高的亩利润。除此之外，相同的农作物在不同的地块类型、种植季次的亩利润也不同，例如黄瓜在水浇地种植的亩利润有 81100 元/亩，在智慧大棚第一季的亩利润有 101500 元/亩，而在智慧大棚第二季的亩利润高达 109550 元/亩。因此，我们在种植时可以参考亩利润这一指标，优先考虑红薯、黄瓜、榆黄菇和大白菜的种植，而且也要考虑不同的地块类型、种植季次对农作物亩利润的影响。

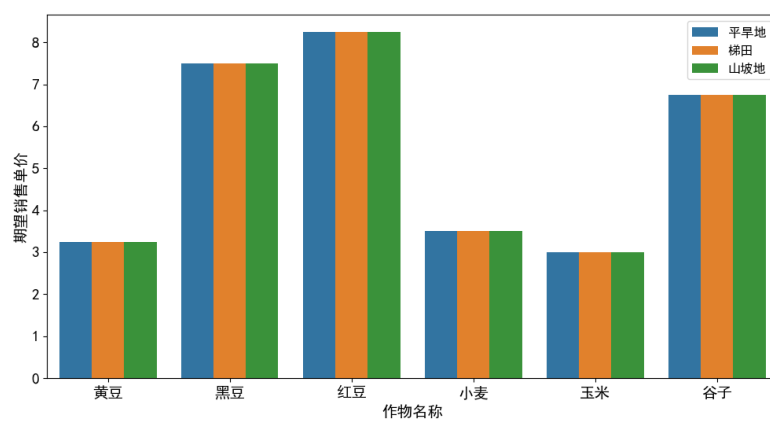
5.2.2 耕地类型及季次对农作物种植及销售的影响

为了进一步探究不同耕地类型、种植季次对农作物种植及销售情况的影响，我们分别绘制出各类农作物在同一季次下不同耕地类型种植对应的亩产量、种植成本及期望销售单价柱状图（探究土地类型影响），同时也绘制出各类农作物在同一耕地类型不同季次下种植对应上述三个统计量的柱状图（探究耕种季次影响）。

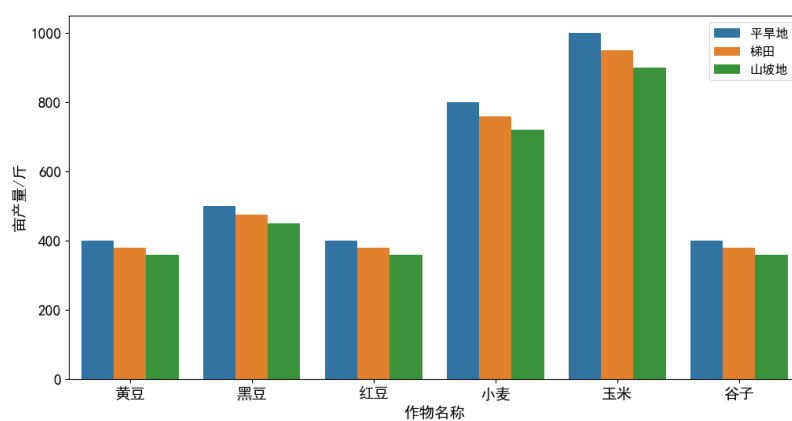
由于数据量较多，在探究土地类型影响时我们仅考虑黄豆、黑豆、红豆、小麦、玉米、谷子分别在平旱地、梯田、山坡地上的种植，以及豇豆、刀豆、芸豆、土豆、西红柿、茄子六种作物分别在水浇地、普通大棚、智慧大棚的种植。绘制得到的柱状图如图3所示（仅列举平旱地、梯田、山坡地相应统计量，其余结果见附录 C）。



(a) 种植成本



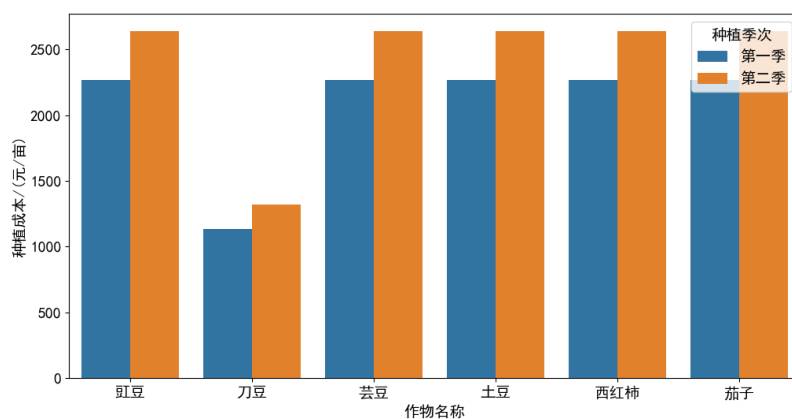
(b) 期望销售单价



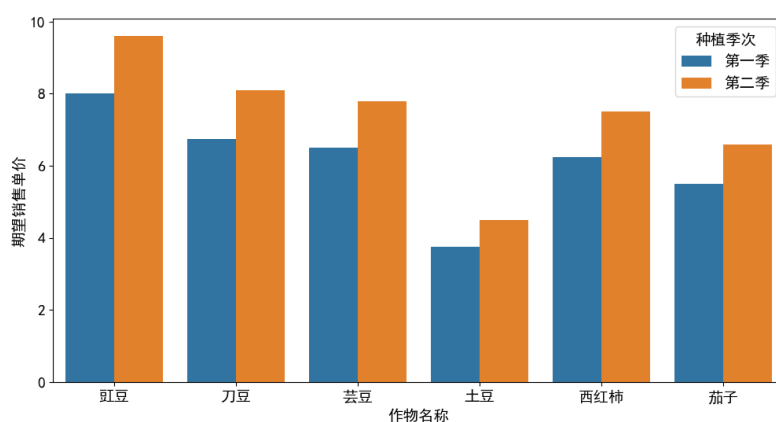
(c) 亩产量

图3 平旱地、梯田、山坡地统计数据

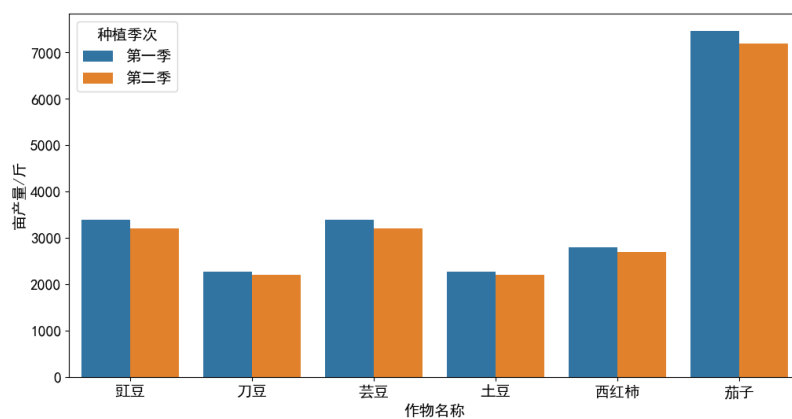
接着我们选择豇豆、刀豆、芸豆、土豆、西红柿、茄子在智慧大棚中的种植情况进行不同季次下三个统计量的值，绘制出柱状图4。



(a) 种植成本



(b) 期望销售单价



(c) 亩产量

图 4 智慧大棚不同季度统计量

根据图表信息，我们作出如下结论：农作物种植在平旱地、梯田、山坡地的地块类型只会影响亩产量；在水浇地、普通大棚和智慧大棚的地块类型中会影响亩产量和种植成本；而农作物的种植季次会同时影响农作物产量、种植成本和销售价格。因此，在后续的建模中我们应该分别考虑地块类型和季度对农作物种植的综合影响。

六、模型建立与求解

6.1 问题 1：农作物种植方案优化模型

6.1.1 模型假设

题目中假设各作物未来的预期销售量、种植成本、销售单价及亩产量都与 2023 年数据相近，为了简化模型，我们在进行实际运算时将其视为相等。此外，题目中也假定各季种植的农作物在当季销售，因此在计算利润时需分别统计各季次的产量、成本、价格及销售量。

6.1.2 数据处理

为了进行销售情况的分析，并将数据用于总利润的计算，我们统计了不同土地类型各季次农作物的亩产量、种植成本、期望销售单价（根据假设 5，取销售单价区间的中值）以及各农作物在不同季次销售的预期销售量（根据假设 6，我们将预期销售量设定为 2023 年对应的总种植产量）。

6.1.3 农作物种植方案非线性规划模型构建

为了减少自变量数目，以简化求解复杂度，我们遍历 2024 年到 2030 年，对每一年单独进行决策方案的求解，同时每一年记录并更新每块耕地累计无豆类作物耕种的年数，通过设立约束条件确保豆类作物根菌的充足。

由于我们对每一年依次单独进行方案求解，所以对于每一年的优化过程来说 t 为常数（若考虑 2024 年，则 $t = 2024$ ）。定义 $L_{all} = \{1, 2, \dots, 54\}$ 为 i 的取值范围，对应全部耕地（顺序与附件 1 中耕地信息一致，即第一、二行数据分别对应 1、2，以次类推）； $C_{all} = \{1, 2, 41\}$ 为 j 的取值范围，对应所有种类总作物（与作物编号一致）； k 的取值范围为 $\{1, 2\}$ ，分别对应第一季与第二季。

（a）决策变量的设立

我们将决策变量设立为 $m_{i,j,k,t}$ ，即年份 t 第 k 季度耕地 i 中作物 j 的种植份数。根据假设 3，我们将大棚土地均分为两份，而非大棚地块的土地视为单独一份。因此对于大棚来说，其取值范围为 $\{0, 1, 2\}$ ，而对于非大棚来说，取值范围为 $\{0, 1\}$ 。具体数学形式可表示为：

$$\begin{aligned} m_{i,j,k,t} &\in \{0, 1, 2\}, (i \in L_H); \\ m_{i,j,k,t} &\in \{0, 1\}, (i \in L_{all} - L_H) \end{aligned} \quad (1)$$

其中 $L_H = \{35, 36, \dots, 54\}$ ，对应大棚地块。

土地耕种份数与耕种面积的转换公式如下：

$$\begin{aligned} x_{i,j,k,t} &= d_i * m_{i,j,k,t} / 2, (i \in L_H); \\ x_{i,j,k,t} &= d_i * m_{i,j,k,t}, (i \in L_{all} - L_H) \end{aligned} \quad (2)$$

(b) 约束条件的确立

1. 根据假设 3 所述，除了无适宜作物可种植的情况外，每个地块的农作物耕种总面积都需要与其耕地面积一致。同时为了便于管理，我们规定单季粮食类作物只在地块的第一季种植。因此若地块在第一季种植了粮食类作物，则该地块第二季应没有任何可耕种面积。

$$\begin{aligned} \sum_{j \in C_{all}} x_{i,j,1,t} &= d_i, (i \in L_{all}) \\ \sum_{j \in C_{all}} x_{i,j,2,t} &= d_i - \sum_{j \in C_S} x_{i,j,1,t}, (i \in L_{all}) \end{aligned} \quad (3)$$

其中 $C_S = \{1, 2, \dots, 16\}$ ，代表单季种植的粮食作物。

2. 由于含有豆类作物根菌的土壤更利于农作物生长，所有土地每三年内需要至少种植一次豆类作物，因此我们选取二年为期限，当某地块累计两年未出现豆类作物的种植，则必须种植满豆类作物（其他时候也可以选择种植豆类以增加利润）。

$$\sum_{j \in C_B} x_{i,j,1,t} > y_i - 1.5, (i \in L_{all}) \quad (4)$$

其中 $C_B = \{1, 2, 3, 4, 5, 17, 18, 19\}$ ，对应豆类作物。当累计未种植豆类年数为 0 或者 1 时，第一季豆类总耕种面积大于一个负数，表示可选择耕种或不耕种；而当累计未种植豆类年数为 2 时，第一季豆类总耕种面积大于 0.5，表示必须进行豆类的耕种，由于我们先前将耕种面积变量进行了离散化，因此该约束条件能保障将某地块（含大棚）种植满。

3. 为了避免重茬种植，即当年第一季作物类型不能与上一年第二季或当年第二季作物类型相同，我们进行以下约束条件：

$$\sum_{j \in C_{all}} x_{i,j,1,t} * (x_{i,j,2,t-1} + x_{i,j,2,t}) = 0, (i \in L_{all}) \quad (5)$$

4. 每种地块在各季度可种植的农作物都有对应的约束范围。

$$\sum_{j \in C_{all} - C_{i,k}} x_{i,j,k,t} = 0, (i \in L_{all}, k \in \{1, 2\}) \quad (6)$$

其中 $C_{i,k}$ 为第 k 季度地块 i 适宜种植的农作物对应的编号，具体取值范围参照表1。

(c) 目标函数的确立

我们以该年农作物总利润最大为目标进行目标函数的设立。首先计算出该年的总种植成本 O_t :

$$O_t = \sum_{k=1}^2 \sum_{i \in L_{all}} \sum_{j \in C_{all}} (x_{i,j,k,t} * ct_{i,j,k,t}) \quad (7)$$

定义中间量 $A_{j,k,t}$, 代表农作物 j 在年份 t 第 k 季度的总产量 (单位为斤), 其计算公式如下:

$$A_{j,k,t} = \sum_{i \in L_{all}} (m_{i,j,k,t} * a_{i,j,k,t}) \quad (8)$$

不同销售情况的区别仅体现在销售总收入的计算上。对于销售情况 1, 即将每季总产量超出相应预期销售量的部分视为滞销, 无任何收入, 对应的总收入计算公式见式9; 而对于销售情况 2, 超出部分根据 2023 年的售价进行 50% 的降价处理, 对应总收入计算公式见式10。

$$I_t = \sum_{k=1}^2 \sum_{j \in C_{all}} (p_{j,k,t} * (\min(s_{j,k,t}, A_{j,k,t}))) \quad (9)$$

$$I_t = \sum_{k=1}^2 \sum_{j \in C_{all}} (p_{j,k,t} * ((\min(s_{j,k,t}, A_{j,k,t}) + 0.5 * \max(A_{j,k,t} - s_{j,k,t}, 0)))) \quad (10)$$

根据总收入减去总成本得到最终的总体农作物利润 W_t :

$$\max W_t = I_t - O_t \quad (11)$$

(d) 非线性规划模型

综上所述给出整体非线性规划模型：

$$\begin{aligned}
 & \max \quad W_t = I_t - O_t \\
 & \text{s.t.} \quad \begin{cases} m_{i,j,k,t} \in Z \\ 0 \leq m_{i,j,k,t} \leq 2, (i \in L_H); \\ 0 \leq m_{i,j,k,t} \leq 1, (i \in L_{all} - L_H) \\ x_{i,j,k,t} = d_i * m_{i,j,k,t} / 2, (i \in L_H); \\ x_{i,j,k,t} = d_i * m_{i,j,k,t}, (i \in L_{all} - L_H) \\ \sum_{j \in C_{all}} x_{i,j,1,t} = d_i, (i \in L_{all}) \\ \sum_{j \in C_{all}} x_{i,j,2,t} = d_i - \sum_{j \in C_S} x_{i,j,1,t}, (i \in L_{all}) \\ \sum_{j \in C_B} x_{i,j,1,t} > y_i - 1.5, (i \in L_{all}) \\ \sum_{j \in C_{all}} x_{i,j,1,t} * (x_{i,j,2,t-1} + x_{i,j,2,t}) = 0, (i \in L_{all}) \\ \sum_{j \in C_{all} - C_{i,k}} x_{i,j,k,t} = 0, (i \in L_{all}, k \in \{1, 2\}) \end{cases} \quad (12)
 \end{aligned}$$

6.1.4 基于约束优化的蒙特卡洛模拟算法

由于规划模型目标函数及部分约束条件仍然比较复杂，难以用传统方法求解，同时考虑到自变量的离散性，我们引入蒙特卡洛模拟算法来求得近似解。蒙特卡洛模拟（Monte Carlo Simulation）是一种通过随机抽样方法来对变量进行赋值的方法，当模拟次数足够多时，即可通过统计分析的方式获取较理想的解，因此该算法可广泛适用于各种复杂问题。

根据先前提炼的非线性规划模型，我们将部分约束条件（称为生成条件）按合适形式转化为对蒙特卡洛随机取值范围的限定，从而提高生成可行解的概率，而其余条件（称为检验条件）则需在生成之后进行检验来判断解合理性。

对于生成条件的设立，我们参考表1数据，设定不同类型时空农作物的挑选范围，结果见表2。除此之外，我们考虑到豆类作物的必要性种植情况，当累计未全面种植豆类作物年数达到两年时，便将农作物挑选范围进一步限定到豆类作物中，以确保结果符合假设 1。

表 2 各时空类型农作物挑选范围

序号	时空类型	农作物挑选范围（编号）
1	平旱地、梯田、山坡地	1, 2, $\dots$, 15
2	水浇地第一季	16, 17, 18, $\dots$, 34
3	水浇地第二季	35, 36, 37（若第一季不种植作物 16）
4	普通大棚、普通大棚第一季	17, 18, $\dots$, 34
5	普通大棚第二季	38, 39, 40, 41
6	智慧大棚第二季	20, 21, $\dots$, 34

对于**检验条件**，我们参考式5，对生成解是否满足不能连续重茬种植条件进行检验。若满足，即可认定为符合假设条件的可行解。

我们使用 Python 设计出蒙特卡洛非线性规划模型求解算法，具体算法步骤如下：

step1 更新累计未全面种植豆类年数：通过读取上一年的相应累计年数与种植情况，对当年累计年数进行更新。

step2 生成满足生成条件的随机解：在生成条件限定的取值范围内随机生成初始解，即各地块（含大棚）中不同农作物的耕种土地份数（根据土地总面积可转化为耕种面积）。

step3 筛选可行解：通过重复第 2 步生成大量的随机初始解，并将符合检验条件的认定为可行解。

step4 计算目标函数值：计算每一个可行解的目标函数值，即当年的农作物总利润。

step5 选取最优解：挑选出最优目标函数值对应的可行解作为我们的优化结果，即最终种植方案。

对于销售情况 1，我们遍历 2024 年到 2030 年，对于每一年使用上述算法随机生成 10000 个初始解进行检验并记录最大农作物总利润，得到各年的最大总利润。销售情况 2 通过改变相应优化目标并使用相同的算法流程得到结果。最大总利润结果可见表3（对应的耕种方案已记录在 result1_1.xlsx 及 result1_2.xlsx 中）。将各年份的利润相加，我们得到第一种销售情况未来七年的总利润约为 29610565.25 元，而第二种情况总利润约为 49422363.75 元。

表 3 问题 1 总利润求解结果

年份	销售情况 1 总利润（元）	销售情况 2 总利润（元）
2024	4434289.75	7336504.50
2025	4012395.25	6184817.63
2026	4219654.25	7714772.63
2027	4182357.75	7225660.00
2028	4171015.25	6236945.25
2029	4275195.25	7590390.38
2030	4315657.75	7133273.38

6.2 问题 2：考虑数据波动的农作物种植方案优化模型

在问题 1 的模型基础上，我们需要进一步考虑农作物亩产量、种植成本、销售单价及销售量的波动情况。因此需要对题目提供的条件进行分析，设计出不同类型数据的波动方式，最后结合问题 1 中的模型进行求解。

6.2.1 随机年增长率生成模型

为了使数据更符合生活实际情况，我们在题目设定的波动范围上引入了截断正态分布 [2] 进行具体的波动数值生成。对于一个均值、标准差分别为 $\bar{\mu}$ 、 $\bar{\sigma}$ 的正态分布，其截断正态分布的概率密度函数 ψ 计算公式如下：

$$\psi(\bar{\mu}, \bar{\sigma}, a, b; x) = \begin{cases} 0 & x \leq a \\ \frac{\varphi(\bar{\mu}, \bar{\sigma}^2; x)}{\Phi(\bar{\mu}, \bar{\sigma}^2; b) - \Phi(\bar{\mu}, \bar{\sigma}^2; a)} & a < x < b \\ 0 & b \leq x \end{cases} \quad (13)$$

其中 $\varphi(\bar{\mu}, \bar{\sigma}^2; x)$ 代表标准正态分布概率密度函数， $\Phi(\bar{\mu}, \bar{\sigma}^2; b)$ 表示标准正态分布概率分布函数， a 和 b 分别代表截断正态分布中随机变量取值的上下界。

我们定义各数据的随机年增长率 r ，且 r 服从一个截断正态分布，其具体数值的生成过程如下：

step1 定义截断正态分布参数：根据题目所给波动范围设定截断区间 $[a, b]$ ，如小麦和玉米预期销售量年平均增长率介于 5%~10%，则 $a = 0.05, b = 0.1$ 。求出数据的样本均值和样本标准差作为 $\bar{\mu}$ 和 $\bar{\sigma}$ 的取值。

step2 标准化截断区间：对截断区间上下界分别进行标准化，公式如下：

$$a' = \frac{a - \bar{\mu}}{\bar{\sigma}}, \quad b' = \frac{b - \bar{\mu}}{\bar{\sigma}} \quad (14)$$

step3 生成标准正态分布随机数：用标准正态分布生成位于标准化截断区间 $[a', b']$ 内的随机数 z 。

step4 生成随机年增长率 r ：通过对 z 进行反标准化得到符合原始分布的随机数 r ：

$$r = \bar{\mu} + \bar{\sigma} * z \quad (15)$$

我们根据题目要求，对提到的各类型数据随机年增长率对应的的截断正态分布参数进行了统计，后续依次为根据随机生成对应年增长率 r ；对于增长率波动区间较小的数据，使用固定年增长率（如每年增长 5% 左右，则年增长率 r 为固定值 0.05）。统计结果如表4所示。

表 4 年增长率相关信息统计

数据类型	$\bar{\mu}$	$\bar{\sigma}$	a	b	固定年增长率
玉米与小麦的预期销售量	0.075	0.025/3	0.05	0.10	\
玉米与小麦之外农作物的预期销售量	0	0.05/3	-0.05	0.05	\
农作物的亩产量	0	0.1/3	-0.10	0.10	\
农作物的种植成本	\	\	\	\	0.05
粮食类作物的销售价格	\	\	\	\	0.00
蔬菜类作物的销售价格	\	\	\	\	0.05
羊肚菌的销售价格	\	\	\	\	-0.05
羊肚菌之外食用菌的销售价格	-0.03	0.02/3	-0.05	-0.01	\

6.2.2 种植方案规划模型建立与求解

对于表4中提及的各种类型数据，我们根据生成的年增长率（包括固定年增长率）对数据进行变动，具体计算公式如下：

$$amount = amount * (1 + r) \quad (16)$$

其中 $amount$ 可代表各种类型的数据， r 代表其对应的年增长率。

我们仅针对销售情况 1 进行模型求解，即超出部分造成滞销浪费。沿用问题 1 中

的模型和求解算法，我们得到各年最大总利润如表5所示（对应的耕种方案已记录在result2.xlsx中）。将各年得到的利润相加，我们得到这七年的总利润约为 30842800.87 元。

表 5 问题 2 总利润求解结果

年份	销售情况 1 总利润（元）
2024	4441547.39
2025	4288805.71
2026	4468021.46
2027	4402287.01
2028	4329031.96
2029	4322881.22
2030	4590226.11

6.3 问题 3：引入相关性因素的农作物种植方案优化模型

6.3.1 农作物间的可替代性与互补性

在实际生产销售情况中，各农作物间可能存在一定的可替代性与互补性。对于农作物来说，两种作物间的可替代性说明它们具有某种类似的性质，在一些功能发挥上可相互代替。例如，高粱和小麦都可用作饲料和主食，因此存在可替代性。而对于农作物间的互补性，我们从种植角度进行分析，两种农作物间的互补性表明对它们进行配合种植可提高农作物的产量。例如，大豆能通过固氮为玉米提供氮素，以提高玉米的产量 [3]。

考虑到相同类型的农作物间更有可能存在可替代性，我们将农作物依据作物类型划分为五种组别，如表6所示。

假设每个组别内各农作物之间存在可替代性，具有联动效应，当某一农作物价格上涨时，其他同组别农作物的需求量便会提高。且两种作物价格相关系数越大，说明可替代性越强，相应影响幅度会更大。为了简化模型，我们假设同组别农作物间的可替换性强度一致。具体计算方式体现为，某一农作物的价格上涨幅度会造成组别中其他农作物预期销售量的提升，提升幅度为价格上涨幅度与某一固定比例之积。我们设立这一固定比例为 $\alpha = 0.1$ 。

考虑到在种植过程中豆类作物与其他作物的互补性较强。参考文献 [3, 4]，我们假设豆类作物可提升同一地块（含大棚）中下轮种植的其他类型农作物的亩产量，约为 25%。

表 6 农作物组别划分表

序号	组别	作物编号范围
1	豆类粮食	1, 2, 3, 4, 5
2	非豆类粮食	6, 7, $\dots$, 16
3	豆类蔬菜	17, 18, 19
4	非豆类蔬菜	20, 21, $\dots$, 37
5	食用菌	38, 39, 40, 41

6.3.2 预期销售量与销售单价、种植成本间的相关性

为了分析预期销售量与种植成本、销售单价的相关性，我们引入灰色关联分析。灰色关联分析通过分析母序列与多个子序列几何形状的差异来分析各因素间的关联性，能有效运用于数据量较少的情况。我们以 2023 年各农作物不同季节的预期销售量为母序列，其对应的种植成本及期望销售单价作为两个子序列，计算灰色关联度，关联度越大，表示序列间的关联性越强。具体的计算步骤如下：

step1 定义母序列 $w_0(k)$ ，即预期销售量序列。定义子序列 $w_i(k)$, ($i \in 1, 2$)，分别代表种植成本序列和销售单价序列。

step2 对序列数据进行统一量纲化，采用均值化方法。

step3 通过下述公式计算母序列与各自序列间对应元素的关联系数：

$$\gamma(w_0(k), w_i(k)) = \frac{\delta_{min} + \rho\delta_{max}}{|w_0(k) - w_i(k)| + \rho\delta_{max}}$$

$$\delta_{min} = \min_i \min_k |w_0(k) - w_i(k)|$$

$$\delta_{max} = \max_i \max_k |w_0(k) - w_i(k)|$$
(17)

其中 ρ 为分辨系数，系数的值越小，不同大小关联系数间代表的差异越大，我们取 0.5 作为分辨系数。

step4 计算各对应元素间关联系数的均值，作为序列间的关联度：

$$r(w_0, w_i) = \frac{1}{n} \sum_{k=1}^n \gamma(w_0(k), w_i(k))$$
(18)

我们使用 SPSSPRO 进行灰色关联分析求解，得到灰色关联度如表7所示。

表 7 灰色关联度结果

评价项	关联度	排名
期望销售单价（元/斤）	0.864	1
种植成本（元/亩）	0.849	2

表7结果表明，期望销售单价与预期销售量之间的关联性最为显著，当期望销售单价上升时，预期销售量呈现出明显的下降趋势，反之亦然。虽然种植成本对预期销售量的影响相对较弱，但是种植成本的增加可能会导致农民减少种植面积或选择成本较低的种植方式，从而影响整体市场供应量，进而对销售量产生间接影响。这一结论与实际情况相符。通过计算出的灰色关联度，我们定义预期销售量的增长率对其他量的作用为：

$$r_s = -(0.864 * r_p + 0.849 * r_c) \quad (19)$$

其中 r_s 代表下一年预期销售量的增长率， r_p 代表销售单价的增长率， r_c 代表种植成本的增长率。

6.3.3 种植方案规划模型建立与求解

在问题 2 求解模型的基础上，我们引入上述因素用蒙特卡洛算法进行求解（为与问题 2 一致，使用销售情况 1 对应目标函数）。在问题 2 基础上引入的因素可总结为以下公式：

$$r_s = R + \alpha r_{sum} - (0.864 * r_p + 0.849 * r_c) \quad (20)$$

其中 r_s 为某农作物的预期销售量变化率， R 为问题 2 考虑因素下造成的变化率， r_{sum} 为该农作物所属组别中的其他农作物价格变化率的总和。

经过迭代得到较优的农作物种植方案及其相应总利润。为了更直观地比较问题 2 及问题 3 的求解结果，我们将各年份的总利润绘制成折线图，如图5所示。

通过对折线图的分析，我们可以观察到，问题 2 与问题 3 的总利润趋势存在显著差异。具体而言，问题 2 的利润结果在约 440 万元左右波动，而在考虑更多实际情况后，问题 3 的利润则呈现出逐年下降的趋势。这种现象可以归因于每年市场状况的不稳定性，例如疫情和国际贸易摩擦等因素，可能会显著降低农作物的利润。此外，随着时间的推移，农作物利润预测的难度也在上升，市场状况越来越不稳定。这一现象符合现实生活中的情况，表明问题 3 的模型比问题 2 的模型更具现实意义。

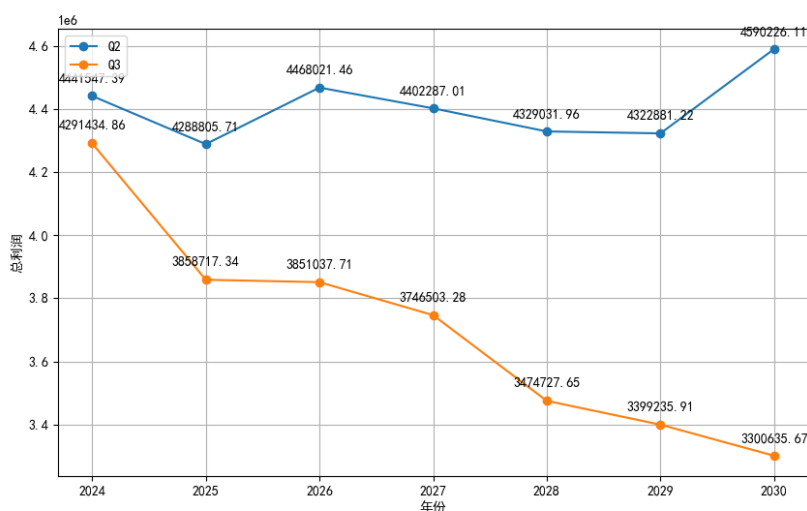


图 5 问题 2 与问题 3 总利润折线图

七、模型的合理性分析和灵敏度分析

7.1 合理性分析

我们对非线性规划的模型与蒙特卡洛模拟的代码求解过程进行约束条件检查，确保所有约束条件在模拟过程中都得到满足。因此我们的模型具有合理性，满足题目的求解要求。

7.1.1 模拟结果的合理性分析

其次，我们对模拟结果进行合理性分析。首先，重新对问题一第一小问进行模型求解，并采用蒙特卡洛模拟算法生成 2024 年的结果表。我们进行了 10000 次迭代，并重复了 10 次实验。每次实验得到的总利润如图6所示。

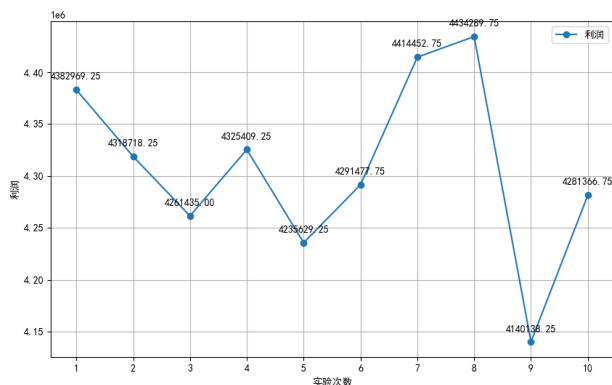


图 6 各次实验得到的总利润

随后，对模拟结果进行了统计分析，计算模拟结果的均值、标准差等统计特性，以确保结果的合理性和实际意义，同时我们进行 Shapiro-Wilk 检验，探究总利润是否符合正态分布，统计与检验结果如表8所示。

表 8 实验总利润统计结果及 Shapiro-Wilk 检验结果

平均值	标准差	中位数	最大值	最小值	检验统计量	检验 p 值
4308588.6	88117.3	4305098.0	4434289.8	4140138.3	0.97	0.88

经过 Shapiro-Wilk 检验，得到 p 值约为 0.88，因此在 0.05 的显著性上，不能拒绝原假设，即数据符合正态分布。此外，我们得到了检验统计量为 0.968，这与 1 非常接近，表明数据分布非常符合正态分布。同时，模拟结果的最大值与最小值之间的差异较小，标准差为 88117，表明模拟结果没有显著的波动性。综上所述，我们的蒙特卡洛模拟结果显示出较小的波动性，总体上符合正态分布，说明模拟结果良好且稳定，且与实际情况相符。

7.1.2 基于约束优化的蒙特卡洛算法的合理性分析

我们还对基于约束优化的蒙特卡洛算法进行合理性分析。我们通过计算进行约束优化后的算法生成可行解的数量来模拟算法的效率，说明优化算法的合理性。以年份为横坐标，可行解的数量为纵坐标作图，分析算法的效率。最终得到结果如图7。

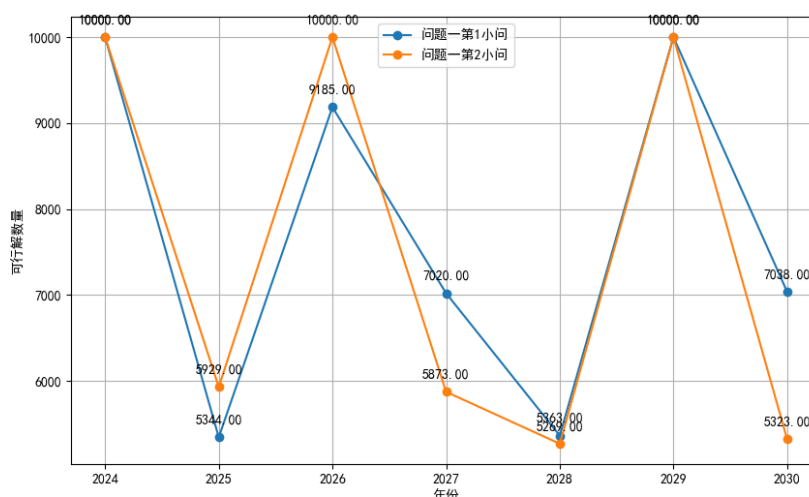


图 7 可行解数量统计

根据折线图 and 具体数据分析，问题一第一小问的平均生成可行解次数为 7485 次，问题二第二小问的平均生成可行解次数为 7692 次。这表明，基于约束优化的蒙特卡洛算

法在生成可行解方面的效率约为 75%，能够有效解决非线性规划问题，并显著提高找到最大利润值的概率。

7.2 敏感性分析

我们对问题 3 模型中农作物价格对其组别内其他作物预期销售量的影响系数 α 进行敏感性分析，分别设立不同 α 的取值，即 0.05、0.10、0.15、0.20。我们以 2024 年的数据重复进行模型求解，得到不同取值下所得最大总利润如表9所示。

表 9 不同 α 取值下的 2024 年最大总利润

α	最大利总润（元）
0.05	4318289.2
0.10	4291434.9
0.15	4330999.5
0.20	4321184.4

通过上述分析，我们可以发现，随着 α 值的增加，模拟利润并没有呈现出明显的单调变化趋势。这表明参数 α 对于模型不敏感性，而且它对利润的影响较为复杂，可能受到其他因素的共同作用。因此，在对模型的优化中，我们需要综合考虑多个参数的相互影响，以获得更为准确和可靠的结果。

八、模型的评价与推广

8.1 模型的优点

1. 本文的非线性规划模型的建立不仅符合题目要求，还具有普适性，可以应用到其他相关的实际问题。
2. 本文采用基于约束优化的蒙特卡洛模拟，可以有效提高出现可行解的概率，收敛速度比普通的蒙特卡洛模拟更快。
3. 本文根据样本数量少的特点使用灰色关联模型进行分析，具有鲁棒性强，适用性广的特点，让分析结果更加可靠。
4. 本文对模型参数进行敏感性分析，使模型评价更加客观可靠。

8.2 模型的缺点

1. 本文的模型是建立在少量数据的基础上进行分析, 缺乏在大量数据训练的过程, 无法保证其拓展性。
2. 灰色关联模型在确定分辨系数时存在一定的主观性, 可能影响结果的客观性。

8.3 模型的推广

1. 我们可以考虑更多的实际情况加入到模型中, 比如: 政策对农作物的需求量与成本的影响, 国际贸易关系对农作物的需求量的影响等。
2. 我们可以通过寻找更多的农作物的实际生产销售数据来完善我们的模型, 验证模型的准确性和可靠性。

参考文献

- [1] 张彪. 玉米重茬的危害与防治措施 [J]. 现代农村科技, 2023, (12): 39-40.
- [2] 张维铭. 截断正态分布的计量抽样方案 [J]. 浙江理工大学学报, 2006, (02): 182-188.
- [3] 李银水, 黄翔, 罗金华, 等. 不同带宽行比对玉米/大豆带状间套作群体产量和氮素效率的影响 [J]. 中国油料作物学报, 2024, 46(4): 863-871.
- [4] 吕玉林. 大豆玉米带状复合种植间作模式与大豆玉米普通间作模式产量比较分析 [J]. 农业科学, 2022, 12(9): 852-856.

附录 A 支撑材料列表

我们生成了支撑材料文件的项目目录树如下：

```
2024C/
  0.创建指定文件夹.py
  1.生成土地可种植农作物总表.py
  2.生成2023年农作物预期销售表.py
  3.生成农作物在不同地块类型的生产情况.py
  4.生成各年蒙特卡洛所需文件.py
  5.生成2023年答案表.py
  Q1.1第一问第一种情况蒙特卡洛算法.py
  Q1.2第一问第二种情况蒙特卡洛算法.py
  Q1.3 绘制可行解图.py
  Q1.4 绘制实验次数对结果影响.py
  Q1.作物_亩利润绘图.py
  Q1.作物_季度绘图.py
  Q1.作物类型-亩产量、种植成本绘图.py
  Q2.1第二问文件迁移.py
  Q2.2生成2024-2030预期销售表.py
  Q2.3生成2024-2030生产销售表.py
  Q2.4 第二问生成蒙特卡洛所需文件.py
  Q2.5 第二问蒙特卡洛.py
  Q3.1第三问文件迁移.py
  Q3.2生成新2024-2030预期产量表.py
  Q3.3第三问生成蒙特卡洛所需文件.py
  Q3.4第三问蒙特卡洛.py
  Q3.5生成灰色关联表格.py
  Q3.6 绘图Q2Q3对比.py
  生成目录树.py
  目录树.txt
  .idea/
    .gitignore
    2024C.iml
    misc.xml
    modules.xml
    workspace.xml
    inspectionProfiles/
      profiles_settings.xml
      Project_Default.xml
  preprocessed_data/
    拆分数据/
      2023年生产销售情况（附件2.2）.CSV
      2023年生产销售情况（附件2.2）.xlsx
      2023年种植情况（附件2.1）.CSV
      2023年种植情况（附件2.1）.xlsx
      作物情况（附件1.2）.xlsx
```


答案表格.xlsx

耕地情况(附件1.1).xlsx

整理数据/

2023年生产销售表(附件2.2).csv

2023年种植情况(附件2.1).CSV

作物名称(附件1.2).CSV

土地可种植农作物总表.CSV

答案总表模板.csv

耕地名称-亩数.csv

Q1/

2023年农作物预期销量表.csv

2023年情况总表.CSV

2023年生产销售表.csv

2023年种植-土地情况.CSV

2024-2030可行解数量.CSV

2024-2030可行解数量.png

2024实验-利润.CSV

2024实验-利润.png

作物1_地块_亩产量.png

作物1_地块_种植成本.png

作物1_地块_销售单价.png

作物2_地块_亩产量.png

作物2_地块_种植成本.png

作物2_地块_销售单价.png

作物_季度_亩产量.png

作物_季度_种植成本.png

作物_季度_销售单价.png

农作物在不同地块类型的生产情况.csv

山坡地_亩利润图.png

平旱地_亩利润图.png

普通大棚_亩利润图.png

智慧大棚_亩利润图.png

梯田_亩利润图.png

水浇地_亩利润图.png

2023/

2023_答案表_1.csv

2023_答案表_2.csv

2024/

2024_产量表.csv

2024_利润_1.txt

2024_利润_1_1.txt

2024_利润_1_10.txt

2024_利润_1_2.txt

2024_利润_1_3.txt

2024_利润_1_4.txt

2024_利润_1_5.txt

2024_利润_1_6.txt

2024_利润_1_7.txt
2024_利润_1_8.txt
2024_利润_1_9.txt
2024_利润_2.txt
2024_单价表.csv
2024_成本表.csv
2024_生产销售表.csv
2024_答案表_1.csv
2024_答案表_2.csv
2024_解_1.csv
2024_解_1_10年解.csv
2024_解_1_1年解.csv
2024_解_1_2年解.csv
2024_解_1_3年解.csv
2024_解_1_4年解.csv
2024_解_1_5年解.csv
2024_解_1_6年解.csv
2024_解_1_7年解.csv
2024_解_1_8年解.csv
2024_解_1_9年解.csv
2024_解_2.csv
2024_销量表.csv
2024_预期销量表.csv
2025/
2025_产量表.csv
2025_利润_1.txt
2025_利润_2.txt
2025_单价表.csv
2025_成本表.csv
2025_生产销售表.csv
2025_答案表_1.csv
2025_答案表_2.csv
2025_解_1.csv
2025_解_2.csv
2025_销量表.csv
2025_预期销量表.csv
2025迭代图.png
2026/
2026_产量表.csv
2026_利润_1.txt
2026_利润_2.txt
2026_单价表.csv
2026_成本表.csv
2026_生产销售表.csv
2026_答案表_1.csv
2026_答案表_2.csv
2026_解_1.csv

2026_解_2.csv
2026_销量表.csv
2026_预期销量表.csv
2026迭代图.png
2027/
2027_产量表.csv
2027_利润_1.txt
2027_利润_2.txt
2027_单价表.csv
2027_成本表.csv
2027_生产销售表.csv
2027_答案表_1.csv
2027_答案表_2.csv
2027_解_1.csv
2027_解_2.csv
2027_销量表.csv
2027_预期销量表.csv
2027迭代图.png
2028/
2028_产量表.csv
2028_利润_1.txt
2028_利润_2.txt
2028_单价表.csv
2028_成本表.csv
2028_生产销售表.csv
2028_答案表_1.csv
2028_答案表_2.csv
2028_解_1.csv
2028_解_2.csv
2028_销量表.csv
2028_预期销量表.csv
2028迭代图.png
2029/
2029_产量表.csv
2029_利润_1.txt
2029_利润_2.txt
2029_单价表.csv
2029_成本表.csv
2029_生产销售表.csv
2029_答案表_1.csv
2029_答案表_2.csv
2029_解_1.csv
2029_解_2.csv
2029_销量表.csv
2029_预期销量表.csv
2029迭代图.png
2030/

2030_产量表.csv
2030_利润_1.txt
2030_利润_2.txt
2030_单价表.csv
2030_成本表.csv
2030_生产销售表.csv
2030_答案表_1.csv
2030_答案表_2.csv
2030_解_1.csv
2030_解_2.csv
2030_销量表.csv
2030_预期销量表.csv
2030迭代图.png

Q2/
2023/
2023年农作物预期销量表.csv
2023年生产销售表.csv
2023年答案表.csv

2024/
2024年_产量表.csv
2024年_单价表.csv
2024年_成本表.csv
2024年_销量表.csv
2024年农作物预期销量表.csv
2024年利润.txt
2024年生产销售表.csv
2024年答案表.csv
2024年解.csv

2025/
2025年_产量表.csv
2025年_单价表.csv
2025年_成本表.csv
2025年_销量表.csv
2025年农作物预期销量表.csv
2025年利润.txt
2025年生产销售表.csv
2025年答案表.csv
2025年解.csv

2026/
2026年_产量表.csv
2026年_单价表.csv
2026年_成本表.csv
2026年_销量表.csv
2026年农作物预期销量表.csv
2026年利润.txt
2026年生产销售表.csv
2026年答案表.csv

2026年解.csv
2027/
2027年_产量表.csv
2027年_单价表.csv
2027年_成本表.csv
2027年_销量表.csv
2027年农作物预期销量表.csv
2027年利润.txt
2027年生产销售表.csv
2027年答案表.csv
2027年解.csv
2028/
2028年_产量表.csv
2028年_单价表.csv
2028年_成本表.csv
2028年_销量表.csv
2028年农作物预期销量表.csv
2028年利润.txt
2028年生产销售表.csv
2028年答案表.csv
2028年解.csv
2029/
2029年_产量表.csv
2029年_单价表.csv
2029年_成本表.csv
2029年_销量表.csv
2029年农作物预期销量表.csv
2029年利润.txt
2029年生产销售表.csv
2029年答案表.csv
2029年解.csv
2030/
2030年_产量表.csv
2030年_单价表.csv
2030年_成本表.csv
2030年_销量表.csv
2030年农作物预期销量表.csv
2030年利润.txt
2030年生产销售表.csv
2030年答案表.csv
2030年解.csv
Q3/
Q2Q3对比.csv
Q2Q3对比.png
灰色关联分析表.csv
2023/
2023年农作物预期销量表.csv

2023年生产销售表.csv
2023年答案表.csv
2024/
2024_10年利润.txt
2024_10年解.csv
2024_1年利润.txt
2024_1年解.csv
2024_2年利润.txt
2024_2年解.csv
2024_3年利润.txt
2024_3年解.csv
2024_4年利润.txt
2024_4年解.csv
2024_5年利润.txt
2024_5年解.csv
2024_6年利润.txt
2024_6年解.csv
2024_7年利润.txt
2024_7年解.csv
2024_8年利润.txt
2024_8年解.csv
2024_9年利润.txt
2024_9年解.csv
2024年_产量表.csv
2024年_单价表.csv
2024年_成本表.csv
2024年_销量表.csv
2024年农作物预期销量表.csv
2024年利润.txt
2024年生产销售表.csv
2024年答案表.csv
2024年解.csv
2025/
2025_10年利润.txt
2025_10年解.csv
2025_1年利润.txt
2025_1年解.csv
2025_2年利润.txt
2025_2年解.csv
2025_3年利润.txt
2025_3年解.csv
2025_4年利润.txt
2025_4年解.csv
2025_5年利润.txt
2025_5年解.csv
2025_6年利润.txt
2025_6年解.csv

2025_7年利润.txt
2025_7年解.csv
2025_8年利润.txt
2025_8年解.csv
2025_9年利润.txt
2025_9年解.csv
2025年_产量表.csv
2025年_单价表.csv
2025年_成本表.csv
2025年_销量表.csv
2025年农作物预期销量表.csv
2025年利润.txt
2025年生产销售表.csv
2025年答案表.csv
2025年解.csv
2026/
2026_10年利润.txt
2026_10年解.csv
2026_1年利润.txt
2026_1年解.csv
2026_2年利润.txt
2026_2年解.csv
2026_3年利润.txt
2026_3年解.csv
2026_4年利润.txt
2026_4年解.csv
2026_5年利润.txt
2026_5年解.csv
2026_6年利润.txt
2026_6年解.csv
2026_7年利润.txt
2026_7年解.csv
2026_8年利润.txt
2026_8年解.csv
2026_9年利润.txt
2026_9年解.csv
2026年_产量表.csv
2026年_单价表.csv
2026年_成本表.csv
2026年_销量表.csv
2026年农作物预期销量表.csv
2026年利润.txt
2026年生产销售表.csv
2026年答案表.csv
2026年解.csv
2027/
2027_10年利润.txt

2027_10年解.csv
2027_1年利润.txt
2027_1年解.csv
2027_2年利润.txt
2027_2年解.csv
2027_3年利润.txt
2027_3年解.csv
2027_4年利润.txt
2027_4年解.csv
2027_5年利润.txt
2027_5年解.csv
2027_6年利润.txt
2027_6年解.csv
2027_7年利润.txt
2027_7年解.csv
2027_8年利润.txt
2027_8年解.csv
2027_9年利润.txt
2027_9年解.csv
2027年_产量表.csv
2027年_单价表.csv
2027年_成本表.csv
2027年_销量表.csv
2027年农作物预期销量表.csv
2027年利润.txt
2027年生产销售表.csv
2027年答案表.csv
2027年解.csv
2028/
2028_10年利润.txt
2028_10年解.csv
2028_1年利润.txt
2028_1年解.csv
2028_2年利润.txt
2028_2年解.csv
2028_3年利润.txt
2028_3年解.csv
2028_4年利润.txt
2028_4年解.csv
2028_5年利润.txt
2028_5年解.csv
2028_6年利润.txt
2028_6年解.csv
2028_7年利润.txt
2028_7年解.csv
2028_8年利润.txt
2028_8年解.csv

2028_9年利润.txt
2028_9年解.csv
2028年_产量表.csv
2028年_单价表.csv
2028年_成本表.csv
2028年_销量表.csv
2028年农作物预期销量表.csv
2028年利润.txt
2028年生产销售表.csv
2028年答案表.csv
2028年解.csv

2029/

2029_10年利润.txt
2029_10年解.csv
2029_1年利润.txt
2029_1年解.csv
2029_2年利润.txt
2029_2年解.csv
2029_3年利润.txt
2029_3年解.csv
2029_4年利润.txt
2029_4年解.csv
2029_5年利润.txt
2029_5年解.csv
2029_6年利润.txt
2029_6年解.csv
2029_7年利润.txt
2029_7年解.csv
2029_8年利润.txt
2029_8年解.csv
2029_9年利润.txt
2029_9年解.csv
2029年_产量表.csv
2029年_单价表.csv
2029年_成本表.csv
2029年_销量表.csv
2029年农作物预期销量表.csv
2029年利润.txt
2029年生产销售表.csv
2029年答案表.csv
2029年解.csv

2030/

2030_10年利润.txt
2030_10年解.csv
2030_1年利润.txt
2030_1年解.csv
2030_2年利润.txt

```
2030_2年解.csv
2030_3年利润.txt
2030_3年解.csv
2030_4年利润.txt
2030_4年解.csv
2030_5年利润.txt
2030_5年解.csv
2030_6年利润.txt
2030_6年解.csv
2030_7年利润.txt
2030_7年解.csv
2030_8年利润.txt
2030_8年解.csv
2030_9年利润.txt
2030_9年解.csv
2030年_产量表.csv
2030年_单价表.csv
2030年_成本表.csv
2030年_销量表.csv
2030年农作物预期销量表.csv
2030年利润.txt
2030年生产销售表.csv
2030年答案表.csv
2030年解.csv
raw_data/
  C题.pdf
  附件1.xlsx
  附件2.xlsx
  附件3/
    result1_1.xlsx
    result1_2.xlsx
    result2.xlsx
答案（附件3）/
  附件3/
    result1_1.xlsx
    result1_2.xlsx
    result2.xlsx
```

附录 B 不同土地类型亩利润统计

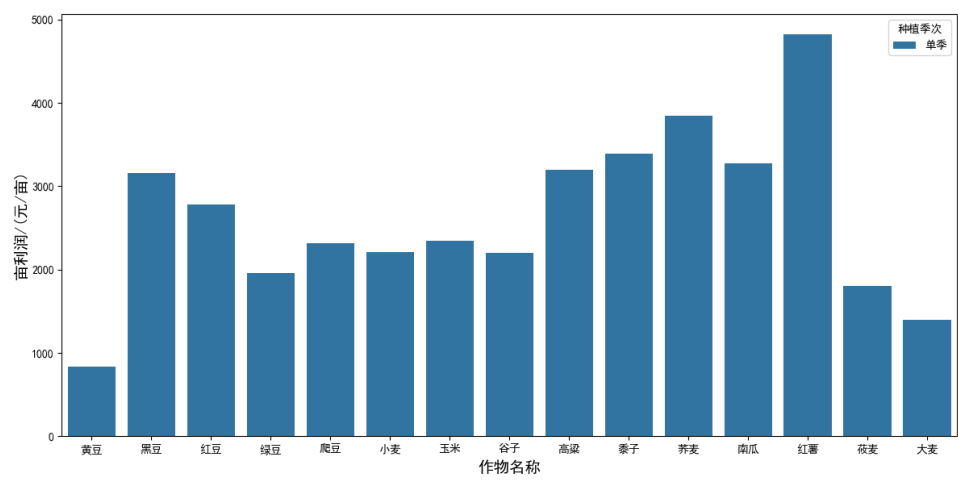


图 8 梯田相应亩产量统计

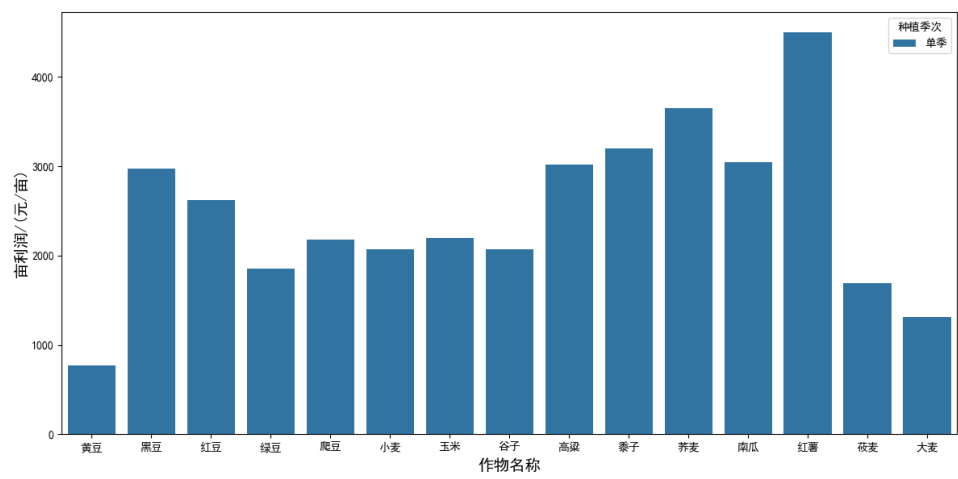


图 9 山坡地相应亩产量统计

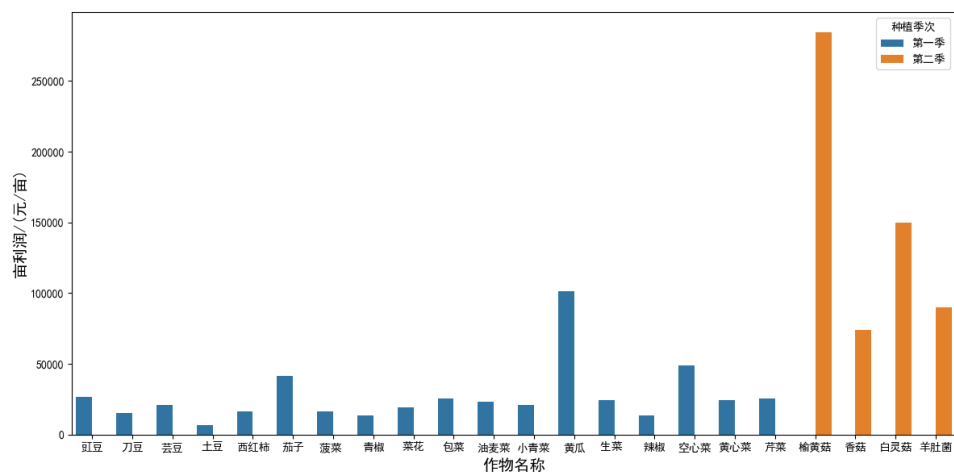


图 10 普通大棚相应亩产量统计

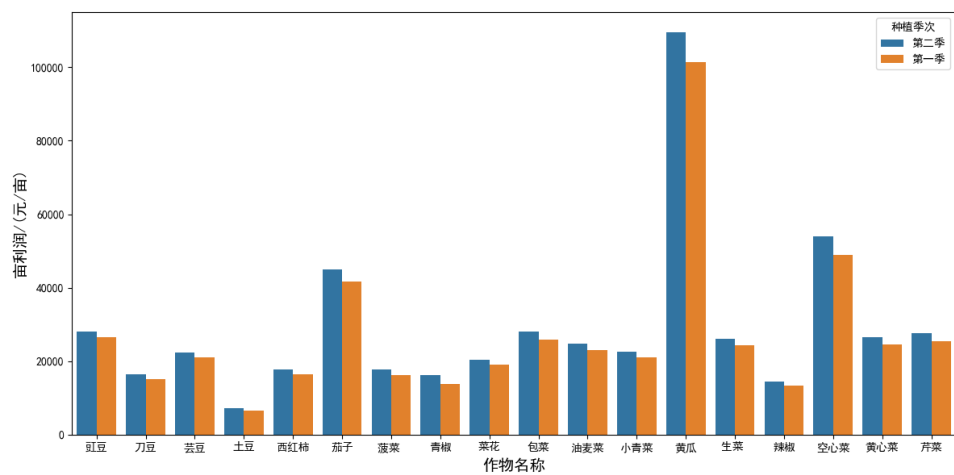


图 11 智慧大棚相应亩产量统计

附录 C 不同土地类型作物亩产量、种植成本、期望销售单价统计

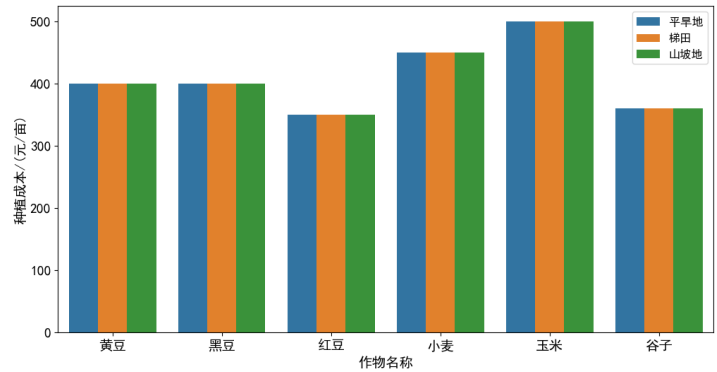


图 12 平旱地、梯田、山坡地不同耕地类型种植成本

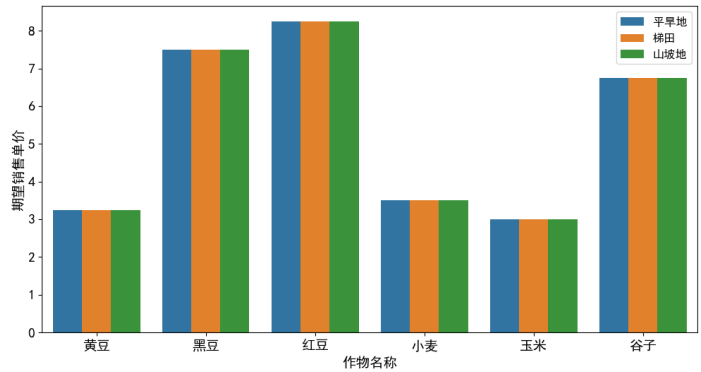


图 13 平旱地、梯田、山坡地不同耕地类型种期望销售单价

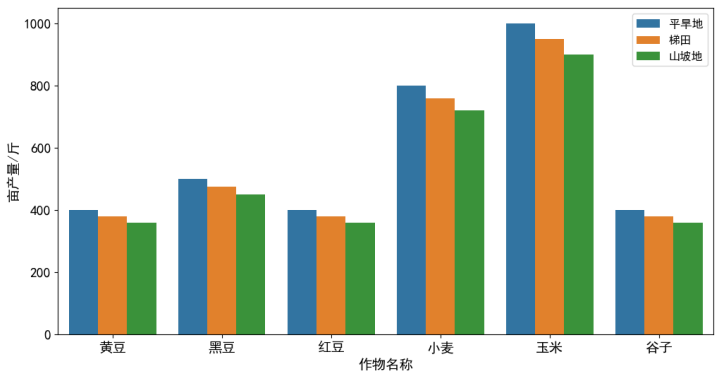


图 14 平旱地、梯田、山坡地不同耕地类型种亩产量

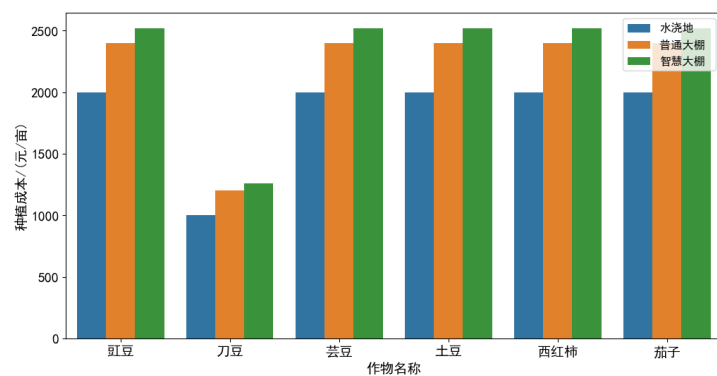


图 15 水浇地、大棚不同耕地类型种植成本

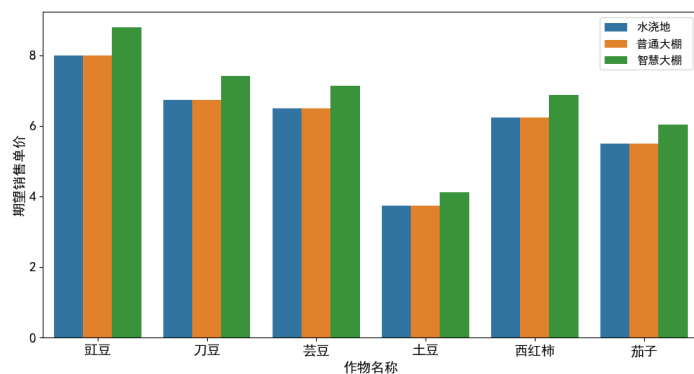


图 16 水浇地、大棚不同耕地类型种期望销售单价

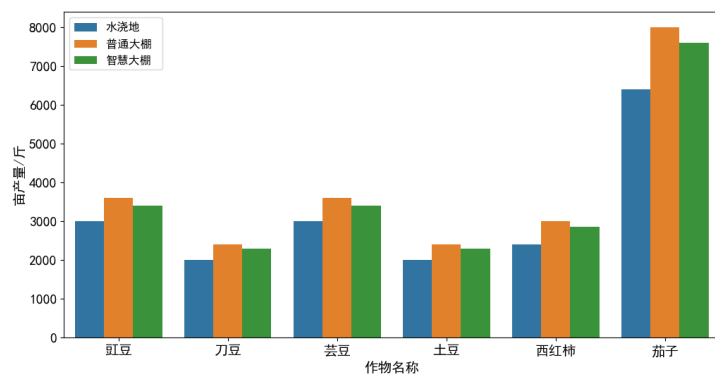


图 17 水浇地、大棚不同耕地类型种亩产量

附录 D 源程序

0. 创建指定文件夹（python 源程序）

```
import os

# 基础路径
base_path = 'preprocessed_data/整理数据'

# 文件夹名称列表
folders = ['Q1', 'Q2', 'Q3']
subfolders = [str(year) for year in range(2023, 2031)]

# 创建文件夹
for folder in folders:
    for subfolder in subfolders:
        path = os.path.join(base_path, folder, subfolder)
        os.makedirs(path, exist_ok=True)
        print(f'文件夹已创建: {path}')
```

1. 生成土地可种植农作物总表（python 源程序）

```
import pandas as pd
data=pd.read_csv("preprocessed_data/拆分数据/2023年生产销售情况（附件2.2）.CSV",encoding='gbk')
data=data.iloc[0:107,0:7]
crop=pd.read_excel("preprocessed_data/拆分数据/作物情况（附件1.2）.xlsx")
crop=crop.iloc[0:41,0:3]
crop_name=crop
print(crop_name)
crop_name.to_csv("preprocessed_data/整理数据/作物名称（附件1.2）.CSV",encoding='gbk',index=False)

df1 = data[['地块类型', '种植季次']].drop_duplicates().reset_index(drop=True)
for crop in crop_name['作物名称']:
    df1[crop] = 0

# 根据 df 来对这些属性赋值 0 或 1
for index, row in data.iterrows():
    df1.loc[(df1['地块类型'] == row['地块类型']) & (df1['种植季次'] == row['种植季次']),
            row['作物名称']] = 1

row_to_copy = df1[(df1['地块类型'] == '普通大棚') & (df1['种植季次'] == '第一季')]
new_row = row_to_copy.copy()
print(new_row)
new_row['地块类型'] = '智慧大棚'
df1 = pd.concat([df1, new_row], ignore_index=True)
```

```
df1.loc[4, '水稻']=1
print(df1)
df1.to_csv("preprocessed_data/整理数据/土地可种植农作物总表.CSV",encoding='gbk',index=False)
```

2. 生成 2023 年农作物预期销售表（python 源程序）

```
import pandas as pd
gengdi=pd.read_excel("preprocessed_data/拆分数据/耕地情况(附件1.1).xlsx")
gengdi=gengdi.iloc[:,0:3]
gengdi.to_csv("preprocessed_data/整理数据/耕地名称-亩数.csv",encoding='gbk',index=False)
gengdi=pd.read_csv("preprocessed_data/整理数据/耕地名称-亩数.csv",encoding='gbk')
gengdi.loc[:, '种植地块']=gengdi.iloc[:,0]

zhongzhi=pd.read_csv("preprocessed_data/拆分数据/2023年种植情况(附件2.1).CSV",encoding='gbk')
# 使用 ffill 方法填充空值
zhongzhi['种植地块'] =zhongzhi['种植地块'].fillna(method='ffill')
zhongzhi.to_csv("preprocessed_data/整理数据/2023年种植情况(附件2.1).CSV",encoding='gbk',index=False)
new_df=pd.merge(gengdi,zhongzhi,on='种植地块')

shengchangxiaoshou=pd.read_csv("preprocessed_data/拆分数据/2023年生产销售情况(附件2.2).CSV",\
encoding='gbk')
shengchangxiaoshou=shengchangxiaoshou.iloc[0:107,: ]
#更新生产销售情况
# 找到地块类型为“普通大棚”且种植季次为“第一季”的数据
rows_to_copy = shengchangxiaoshou[(shengchangxiaoshou['地块类型'] == '普通大棚') &
    (shengchangxiaoshou['种植季次'] == '第一季')]

new_rows = rows_to_copy.copy()
# 修改地块类型为“智慧大棚”
new_rows['地块类型'] = '智慧大棚'
# 将新数据添加到 df 中
shengchangxiaoshou = pd.concat([shengchangxiaoshou, new_rows], ignore_index=True)
shengchangxiaoshou=shengchangxiaoshou.iloc[:,-2]

# 计算确切销售单价
shengchangxiaoshou['确切销售单价/(元/斤)'] = shengchangxiaoshou['销售单价/(元/斤)'].apply(lambda
    x: (float(x.split('-')[0]) + float(x.split('-')[1])) / 2)

# 计算亩利润
shengchangxiaoshou['亩利润/(元/亩)'] = shengchangxiaoshou['亩产量/斤'] *
    shengchangxiaoshou['确切销售单价/(元/斤)'] - shengchangxiaoshou['种植成本/(元/亩)']

shengchangxiaoshou.to_csv("preprocessed_data/整理数据/2023年生产销售表(附件2.2).csv",\
```



```

encoding='gbk')
shengchangxiaoshou.to_csv("preprocessed_data/整理数据/Q1/2023年生产销售表.csv",\
encoding='gbk',index=False)

new_df.to_csv("preprocessed_data/整理数据/Q1/2023年种植-土地情况.CSV",encoding='gbk',index=False)
# 将需要合并的列转换为相同类型
new_df['作物编号'] = new_df['作物编号'].astype(str)
shengchangxiaoshou['作物编号'] = shengchangxiaoshou['作物编号'].astype(str)

total_df=pd.merge(new_df,shengchangxiaoshou,on=['作物名称','地块类型','种植季次','作物编号'])

# 计算每一组中的“产量”值
total_df['产量'] = total_df['种植面积/亩'] * total_df['亩产量/斤']
total_df.to_csv("preprocessed_data/整理数据/Q1/2023年情况总表.CSV",encoding='gbk',index=False)

# 按照作物名称和种植季次来分组，统计产量
grouped_df = total_df.groupby(['作物名称', '种植季次']).agg({'产量': 'sum'}).reset_index()

# 将作物编号添加到新的 DataFrame 中
grouped_df = grouped_df.merge(total_df[['作物名称', '作物编号']].drop_duplicates(),
    on='作物名称', how='left')

look_data=pd.read_csv("preprocessed_data/整理数据/Q1/2023年生产销售表.csv",encoding='gbk')
look_data['作物编号']=look_data['作物编号'].astype(int)
grouped_df['作物编号']=grouped_df['作物编号'].astype(int)
# 检查作物编号在 [17, 34] 之间的记录
for crop_id in range(17, 35):
    if not ((grouped_df['作物编号'] == crop_id) & (grouped_df['种植季次'] == '第二季')).any():
        # 查找对应的亩产量
        yield_per_acre = look_data[(look_data['作物编号'] == crop_id) & (look_data['种植季次']
            == '第二季')]['亩产量/斤'].values[0]
        # 计算新的产量
        new_yield = yield_per_acre * 0.3
        # 获取作物名称
        crop_name = grouped_df[grouped_df['作物编号'] == crop_id]['作物名称'].values[0]
        # 增加新的记录
        new_record = {'作物编号': crop_id, '作物名称': crop_name, '种植季次': '第二季', '产量':
            new_yield}
        grouped_df = grouped_df.append(new_record, ignore_index=True)

# 按作物编号升序排序
grouped_df = grouped_df.sort_values(by=['作物编号','种植季次']).reset_index(drop=True)

print(grouped_df)
#
grouped_df.to_csv("preprocessed_data/整理数据/Q1/2023年农作物预期销量表.csv",encoding='gbk',\

```

```
index=False)
```

3. 生成农作物在不同地块类型的生产情况（python 源程序）

```
import pandas as pd
import matplotlib.pyplot as plt
data=pd.read_csv("preprocessed_data/整理数据/2023年生产销售表(附件2.2).csv",encoding='gbk')
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
# 按作物名称和地块类型分组，计算平均值
grouped =data.groupby(['作物名称', '地块类型']).mean().reset_index()
print(grouped)
grouped.to_csv("preprocessed_data/整理数据/Q1/农作物在不同地块类型的生产情况.csv",\
encoding='gbk',index=False)
```

4. 生成各年蒙特卡洛所需文件（python 源程序）

```
import pandas as pd
import cvxpy
ans_sheet=pd.read_excel("preprocessed_data/拆分数据/答案表格.xlsx")
ans_sheet=ans_sheet.iloc[0:82,1:]

#增加季度属性
# 找到第一个 'F4' 的索引
index_F4 = ans_sheet[ans_sheet['地块名'] == 'F4'].index[0]
# 在第一个 'F4' 之前的行（包含 'F4'）设为 '第一季'
ans_sheet.loc[:index_F4, '季度'] = '第一季'
# 在第一个 'F4' 之后的行设为 '第二季'
ans_sheet.loc[index_F4+1:, '季度'] = '第二季'

#增加亩数和土地类型属性
gengdi=pd.read_csv("preprocessed_data/整理数据/耕地名称-亩数.csv",encoding='gbk')
gengdi_new=gengdi.rename(columns={'地块名称': '地块名'},inplace=False)
ans_total_sheet=pd.merge(ans_sheet,gengdi_new,on='地块名',how='left')

#增加没有种植豆类年数属性
ans_total_sheet.loc[:,'没有种植豆类年数']=0

ans_total_sheet.to_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk',index=False)
shengchangqingkuang=pd.read_csv("preprocessed_data/整理数据/Q1/2023年生产销售表.csv",\
encoding='gbk')
yuqixiaoshou=pd.read_csv("preprocessed_data/整理数据/Q1/2023年农作物预期销量表.csv",\
encoding='gbk')
```

```

for year in range(2024,2031):
    ans_total_sheet.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_1.csv",\
        encoding='gbk',index=False)
    ans_total_sheet.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_2.csv",\
        encoding='gbk', index=False)
    shengchangqingkuang.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_生产销售表.csv",\
        encoding='gbk',index=False)
    yuqixiaoshou.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_预期销量表.csv",\
        encoding='gbk',index=False)

#销量表
yuqixiaoshou['种植季次'] =yuqixiaoshou['种植季次'].replace('单季', '第一季')
pivot_df = yuqixiaoshou.pivot_table(index='种植季次', columns='作物名称', values='产量',
    aggfunc='sum')
pivot_df.reset_index(inplace=True)
pivot_df.fillna(0,inplace=True)
crop_order = yuqixiaoshou[['作物名称',
    '作物编号']].drop_duplicates().sort_values('作物编号')['作物名称'].tolist()
pivot_df = pivot_df[['种植季次'] + crop_order]
pivot_df.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_销量表.csv",encoding='gbk',\
    index=False)

#单价表
shengchangqingkuang['种植季次']=shengchangqingkuang['种植季次'].replace('单季','第一季')
shengchangqingkuang['种植季次'] = shengchangqingkuang['种植季次'].replace('单季', '第一季')
df_unique = shengchangqingkuang.drop_duplicates(subset=['种植季次', '作物名称'])
danjia_df = df_unique.pivot(index='种植季次', columns='作物名称', values='销售单价/(元/斤)')
danjia_df = danjia_df.reindex(
    sorted(danjia_df.columns, key=lambda x:
        shengchangqingkuang[shengchangqingkuang['作物名称'] == x]['作物编号'].values[0]),
    axis=1)
# 定义一个函数来计算区间的平均值
def calculate_average(value):
    if isinstance(value, str) and '-' in value:
        low, high = map(float, value.split('-'))
        return (low + high) / 2
    return value
# 应用函数到 DataFrame 的每个元素
danjia_df = danjia_df.applymap(calculate_average)
danjia_df.fillna(0,inplace=True)
danjia_df.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_单价表.csv", encoding='gbk',
    index=False)

```

```

#成本表
df2=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
df2.fillna(0,inplace=True)
df1=pd.read_csv(f'preprocessed_data/整理数据/Q1/{year}/{year}_生产销售表.csv',encoding='gbk')
df1['种植季次']=df1['种植季次'].replace('单季', '第一季')
df1.rename(columns={'种植季次': '季度'}, inplace=True)
for index, row in df2.iterrows():

    matching_records = df1[(df1['地块类型'] == row['地块类型']) & (df1['季度'] ==
        row['季度'])]
    for _, record in matching_records.iterrows():
        crop_name = record['作物名称']
        planting_cost = record['种植成本/(元/亩)']
        if crop_name in df2.columns:
            df2.at[index, crop_name] = planting_cost

# 保存更新后的 df2
df2.to_csv(f'preprocessed_data/整理数据/Q1/{year}/{year}_成本表.csv', encoding='gbk',
    index=False)

#产量表
df2=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
df2.fillna(0,inplace=True)
df1=pd.read_csv(f'preprocessed_data/整理数据/Q1/{year}/{year}_生产销售表.csv',encoding='gbk')
df1['种植季次']=df1['种植季次'].replace('单季', '第一季')
df1.rename(columns={'种植季次': '季度'}, inplace=True)
for index, row in df2.iterrows():
    matching_records = df1[(df1['地块类型'] == row['地块类型']) & (df1['季度'] ==
        row['季度'])]
    for _, record in matching_records.iterrows():
        crop_name = record['作物名称']
        mu_chanliang = record['亩产量/斤']
        if crop_name in df2.columns:
            df2.at[index, crop_name] = mu_chanliang

# 保存更新后的 df2
df2.to_csv(f'preprocessed_data/整理数据/Q1/{year}/{year}_产量表.csv', encoding='gbk',
    index=False)

```

5. 生成 2023 年答案表（python 源程序）

```

import pandas as pd

temple=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
temple.iloc[:,1:42]=0

```

```

zhongzhiqingkuang=pd.read_csv("preprocessed_data/整理数据/Q1/2023年情况总表.CSV",encoding='gbk')
# 将 "种植季次" 为 "单季" 的改为 "第一季", 并重命名列为 "季度"
zhongzhiqingkuang['种植季次'] =zhongzhiqingkuang['种植季次'].replace('单季', '第一季')
zhongzhiqingkuang.rename(columns={'种植季次': '季度'}, inplace=True)
# 遍历 df2 每一行, 更新作物名称列的值
for index, row in temple.iterrows():
    matching_records = zhongzhiqingkuang[(zhongzhiqingkuang['季度'] == row['季度']) &
        (zhongzhiqingkuang['地块名称'] == row['地块名'])]
    if not matching_records.empty:
        for _, record in matching_records.iterrows():
            temple.at[index, record['作物名称']] = record['种植面积/亩']
print(temple)
temple.to_csv("preprocessed_data/整理数据/Q1/2023/2023_答案表_1.csv",encoding='gbk',index=False)
temple.to_csv("preprocessed_data/整理数据/Q1/2023/2023_答案表_2.csv",encoding='gbk',index=False)

```

Q1.1 第一问第一种情况蒙特卡洛算法（python 源程序）

```

import random
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

bean_index_list = [1, 2, 3, 4, 5, 17, 18, 19]
shuidao_index = 16
shuijiaodi_index = ['D1', 'D2', 'D3', 'D4', 'D5', 'D6', 'D7', 'D8']
LH_index = ['E1', 'E2', 'E3', 'E4', 'E5', 'E6', 'E7', 'E8', 'E9', 'E10', 'E11', 'E12', 'E13',
    'E14', 'E15', 'E16', 'F1', 'F2', 'F3', 'F4']
max_iter=10000

np.random.seed(42)
random.seed(42)

for year in range(2024, 2031):
    # 得到没有种植豆类年数
    year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_1.csv",
        encoding='gbk')
    last_year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year-1}/{year-1}_答案表_1.csv",
        encoding='gbk')
    year_ans.iloc[:, 1:42] = 0
    for index, row in year_ans.iterrows():
        # 获取对应的 last_year_ans 行
        last_year_row = last_year_ans[
            (last_year_ans['地块名'] == row['地块名']) & (last_year_ans['季度'] == row['季度'])]
        if not last_year_row.empty:
            last_year_row = last_year_row.iloc[0]
            # 获取 [1:42] 列范围内不在 bean_index_list 中的列索引
            non_bean_columns = [col for col in range(1, 42) if col not in bean_index_list]

```

```

        if (last_year_row[non_bean_columns] == 0).all():
            year_ans.at[index, '没有种植豆类年数'] = 0
        else:
            year_ans.at[index, '没有种植豆类年数'] = last_year_row['没有种植豆类年数'] + 1

year_ans.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_1.csv", encoding='gbk',
               index=False)
max_profit=0
pass_num=0
print(year_ans.iloc[50,44])

def generate():
    random.seed(42)
    x = np.zeros((82,41))
    #遍历前25行
    for i in range(0,26):
        # 随机选择第2到第16列中的一个列索引
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(1, 6)
        else:
            col_index = np.random.randint(1, 16)
        # 将year_ans第i行的'地块面积/亩'的数值赋值给x的相应位置
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(26, 34):
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(17, 20)
        else:
            col_index = np.random.randint(16, 35)
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(34, 54):
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(17, 20)
            x[i, col_index - 1] = year_ans.iloc[i, 44]
        else:
            col_indices = np.random.choice(range(17, 35), 2, replace=False)
            if np.random.rand() > 0.5:
                # 第一种情况：两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
                x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
                x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
            else:
                # 第二种情况：一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
                x[i, col_indices[0]-1] = year_ans.iloc[i,44]
                x[i, col_indices[1]-1] = 0
    for i in range(54, 62):
        if x[i - 28, 16] != 0:
            x[i, :] = 0
        else:

```

```

        col_index = np.random.randint(35, 38)
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(62, 78):
        col_indices = np.random.choice(range(38, 42), 2, replace=False)
        if np.random.rand() > 0.5:
            # 第一种情况: 两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
            x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
        else:
            # 第二种情况: 一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
            x[i, col_indices[0]-1] = year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0
    for i in range(78, 82):
        col_indices = np.random.choice(range(20, 35), 2, replace=False)
        if np.random.rand() > 0.5:
            # 第一种情况: 两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
            x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
        else:
            # 第二种情况: 一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
            x[i, col_indices[0]-1] = year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0
    return x

def evaluate(x):
    if (constraint4(x)):
        return True
    else:
        return False

def objective(x):
    # 将 x 重新整形为二维数组
    x = x.reshape((year_ans.shape[0], 41))
    xiaoliang = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_销量表.csv",
        encoding='gbk').iloc[:,
        1:].values
    danjia = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_单价表.csv",
        encoding='gbk').values
    chengben = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_成本表.csv",
        encoding='gbk').iloc[:,
        1:42].values
    chanliang=pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_产量表.csv",
        encoding='gbk').iloc[:,1:42].values
    # 确保 danjia 和 chengben 中的所有数据都是数值型
    danjia = danjia.astype(float)
    chengben = chengben.astype(float)
    mu_x=x.copy()

```

```

x=x*chanliang
# 计算 x 的列和
x_sum_1 = np.sum(x[0:54, :], axis=0)
x_sum_2 = np.sum(x[54:, :], axis=0)
# 对每一列进行 min 操作并计算销售金额
x_min_1 = np.minimum(x_sum_1, xiaoliang[0, :])
x_min_2 = np.minimum(x_sum_2, xiaoliang[1, :])
xiaoshou_jine = np.sum(x_min_1 * danjia[0, :]) + np.sum(x_min_2 * danjia[1, :])
# 计算成本
chengben_jine = np.multiply(mu_x, chengben)
# 计算利润
profit = xiaoshou_jine - np.sum(chengben_jine)

return profit
def constraint4(x):
    x = x.reshape((year_ans.shape[0], 41))
    constraints = []
    yijidu_list = [i for i in range(26, 54)]
    for i in yijidu_list:
        for j in range(1, 42):

            aij = float(last_year_ans.iloc[i+28, j])
            bij = float(year_ans.iloc[i + 28, j])
            if ((x[i, j - 1] * (aij + bij)) != 0):
                return False
    return True

if year==2024:
    for j in range(1,11):
        profit=0
        pass_num_1=0
        for i in range(max_iter):
            x = generate()
            if (evaluate(x)):
                pass_num = pass_num + 1
                pass_num_1=pass_num_1+1
                new_profit = objective(x)
                if new_profit > max_profit:
                    print(year, ":", "new_profit", new_profit)
                    pd.DataFrame(x).to_csv\
                        (f"preprocessed_data/整理数据/Q1/{year}/{year}_解_1.csv", encoding='gbk',
                         index=False)
                    max_profit = new_profit
                if new_profit>profit:
                    profit=new_profit
                    pd.DataFrame(x).to_csv\

```



```

        (f"preprocessed_data/整理数据/Q1/{year}/{year}_解_1_{j}年解.csv",
         encoding='gbk',
         index=False)
    with open(f"preprocessed_data/整理数据/Q1/{year}/{year}_利润_1_{j}.txt", 'w',
              encoding='utf-8') as file:
        file.write(str(profit))
        file.write('\n')
        file.write(str(pass_num_1))
else:
    profit_history = []
    for i in range(max_iter):
        x=generate()
        if(evaluate(x)):
            pass_num=pass_num+1
            new_profit=objective(x)
            if new_profit>max_profit:
                print(year,":", "new_profit",new_profit)
                pd.DataFrame(x).to_csv\
                (f"preprocessed_data/整理数据/Q1/{year}/{year}_解_1.csv",encoding='gbk',\
                 index=False)
                max_profit=new_profit
            profit_history.append(max_profit)
    plt.plot(range(max_iter), profit_history)
    plt.xlabel('Iteration')
    plt.ylabel('Max Profit')
    plt.savefig(f"preprocessed_data/整理数据/Q1/{year}/{year}迭代图")
    plt.close()

with open(f"preprocessed_data/整理数据/Q1/{year}/{year}_利润_1.txt", 'w', encoding='utf-8')
    as file:
        file.write(str(max_profit))
        file.write('\n')
        file.write(str(pass_num))
jie=pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_解_1.csv",encoding='gbk')
ans=pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_1.csv",encoding='gbk')
ans.iloc[:, 1:42] = jie.values
ans.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_1.csv",encoding='gbk',\
           index=False)

```

Q1.2 第一问第二种情况蒙特卡洛算法（python 源程序）

```

import random

import pandas as pd
import numpy as np

```

```

bean_index_list = [1, 2, 3, 4, 5, 17, 18, 19]
shuidao_index = 16
shuijiaodi_index = ['D1', 'D2', 'D3', 'D4', 'D5', 'D6', 'D7', 'D8']
LH_index = ['E1', 'E2', 'E3', 'E4', 'E5', 'E6', 'E7', 'E8', 'E9', 'E10', 'E11', 'E12', 'E13',
            'E14', 'E15', 'E16', 'F1', 'F2', 'F3', 'F4']
max_iter=10000
random.seed(42)
np.random.seed(42)

for year in range(2024, 2031):
    # 得到没有种植豆类年数
    year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_2.csv",
                          encoding='gbk')
    last_year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year-1}/{year-1}_答案表_2.csv",
                                encoding='gbk')
    year_ans.iloc[:, 1:42] = 0
    for index, row in year_ans.iterrows():
        # 获取对应的 last_year_ans 行
        last_year_row = last_year_ans[
            (last_year_ans['地块名'] == row['地块名']) & (last_year_ans['季度'] == row['季度'])]
        if not last_year_row.empty:
            last_year_row = last_year_row.iloc[0]
            non_bean_columns = [col for col in range(1, 42) if col not in bean_index_list]
            if (last_year_row[non_bean_columns] == 0).all():
                year_ans.at[index, '没有种植豆类年数'] = 0
            else:
                year_ans.at[index, '没有种植豆类年数'] = last_year_row['没有种植豆类年数'] + 1

    year_ans.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_2.csv", encoding='gbk',
                   index=False)
    max_profit=0
    pass_num=0
    print(year_ans.iloc[50,44])

def generate():
    random.seed(42)
    x = np.zeros((82,41))
    #遍历前25行
    for i in range(0,26):
        # 随机选择第2到第16列中的一个列索引
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(1, 6)
        else:
            col_index = np.random.randint(1, 16)
        # 将year_ans第i行的'地块面积/亩'的数值赋值给x的相应位置
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(26, 34):

```

```

if year_ans.iloc[i,45]>=2:
    col_index=np.random.randint(17, 20)
else:
    col_index = np.random.randint(16, 35)
x[i, col_index-1] = year_ans.iloc[i,44]
for i in range(34, 54):
    if year_ans.iloc[i,45]>=2:
        col_index=np.random.randint(17, 20)
        x[i, col_index - 1] = year_ans.iloc[i, 44]
    else:
        col_indices = np.random.choice(range(17, 35), 2, replace=False)
        if np.random.rand() > 0.5:
            # 第一种情况: 两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
            x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
        else:
            # 第二种情况: 一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
            x[i, col_indices[0]-1] = year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0
for i in range(54, 62):
    if x[i - 28, 16] != 0:
        x[i, :] = 0
    else:
        col_index = np.random.randint(35, 38)
        x[i, col_index-1] = year_ans.iloc[i,44]
for i in range(62, 78):
    col_indices = np.random.choice(range(38, 42), 2, replace=False)
    if np.random.rand() > 0.5:
        # 第一种情况: 两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
        x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
        x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
    else:
        # 第二种情况: 一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
        x[i, col_indices[0]-1]= year_ans.iloc[i,44]
        x[i, col_indices[1]-1] = 0
for i in range(78, 82):
    col_indices = np.random.choice(range(20, 35), 2, replace=False)
    if np.random.rand() > 0.5:
        # 第一种情况: 两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
        x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
        x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
    else:
        # 第二种情况: 一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
        x[i, col_indices[0]-1] = year_ans.iloc[i,44]
        x[i, col_indices[1]-1] = 0
return x

```

```

def constraint4(x):
    x = x.reshape((year_ans.shape[0], 41))
    constraints = []
    yijidu_list = [i for i in range(26, 54)]
    for i in yijidu_list:
        for j in range(1, 42):
            aij = float(last_year_ans.iloc[i+28, j])
            bij = float(year_ans.iloc[i + 28, j])
            if ((x[i, j - 1] * (aij + bij)) != 0):
                return False
    return True

def evaluate(x):
    if (constraint4(x)):
        return True
    else:
        return False

def objective(x):
    # 将 x 重新整形为二维数组
    x = x.reshape((year_ans.shape[0], 41))

    xiaoliang = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_销量表.csv",
        encoding='gbk').iloc[:,
        1:].values
    danjia = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_单价表.csv",
        encoding='gbk').values
    chengben = pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_成本表.csv",
        encoding='gbk').iloc[:,
        1:42].values
    chanliang=pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_产量表.csv",
        encoding='gbk').iloc[:,1:42].values
    # 确保 danjia 和 chengben 中的所有数据都是数值型
    danjia = danjia.astype(float)
    chengben = chengben.astype(float)

    mu_x=x.copy()

    x=x*chanliang
    # 计算 x 的列和
    x_sum_1 = np.sum(x[0:54, :], axis=0)
    x_sum_2 = np.sum(x[54:, :], axis=0)

```

```

# 对每一列进行 min 操作并计算销售金额
x_min_1 = np.minimum(x_sum_1, xiaoliang[0, :])
x_min_2 = np.minimum(x_sum_2, xiaoliang[1, :])

x_diff_1=x_sum_1-xiaoliang[0,:]
x_diff_2=x_sum_2-xiaoliang[1,:]
pd.set_option('display.max_rows', None)
x_diff_1=np.maximum(x_diff_1,0)
x_diff_2=np.maximum(x_diff_2,0)
xiaoshou_jine = np.sum(x_min_1 * danjia[0, :]) + np.sum(x_min_2 * danjia[1, :])
               :]+0.5*np.sum(x_diff_1* danjia[0, :])+0.5*np.sum(x_diff_2* danjia[1, :])
# 计算成本
chengben_jine = np.multiply(mu_x, chengben)
# 计算利润
profit = xiaoshou_jine - np.sum(chengben_jine)

return profit

for i in range(max_iter):
    x=generate()
    if(evaluate(x)):
        pass_num=pass_num+1
        new_profit=objective(x)
        if new_profit>max_profit:
            print(year,":","new_profit",new_profit)
            pd.DataFrame(x).to_csv\
                (f"preprocessed_data/整理数据/Q1/{year}/{year}_解_2.csv",encoding='gbk',index=False)
            max_profit=new_profit

with open(f"preprocessed_data/整理数据/Q1/{year}/{year}_利润_2.txt", 'w', encoding='utf-8')
    as file:
        file.write(str(max_profit))
        file.write('\n')
        file.write(str(pass_num))
jie=pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_解_2.csv",encoding='gbk')
ans=pd.read_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_2.csv",encoding='gbk')
# 打印形状以进行调试
ans.iloc[:, 1:42] = jie.values

ans.to_csv(f"preprocessed_data/整理数据/Q1/{year}/{year}_答案表_2.csv",encoding='gbk',\
index=False)

```

Q1.3. 绘制可行解图（python 源程序）

```
import pandas as pd
```

```

import matplotlib.pyplot as plt
df=pd.read_csv("preprocessed_data/整理数据/Q1/2024-2030可行解数量.CSV")
plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像时负号 '-' 显示为方块的问题
print(df)

# 将第一行作为Q2, 第二行作为Q3
df.index = ['问题一第1小问', '问题一第2小问']

# 转置DataFrame
df = df.T

# 绘制折线图
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['问题一第1小问'], marker='o', label='问题一第1小问')
plt.plot(df.index, df['问题一第2小问'], marker='o', label='问题一第2小问')
for i in range(len(df)):
    plt.annotate(f'{df["问题一第1小问"][i]:.2f}', (df.index[i], df["问题一第1小问"][i]),
        textcoords="offset points", xytext=(0,10), ha='center')
    plt.annotate(f'{df["问题一第2小问"][i]:.2f}', (df.index[i], df["问题一第2小问"][i]),
        textcoords="offset points", xytext=(0,10), ha='center')

plt.xlabel('年份')
plt.ylabel('可行解数量')
plt.legend()

# 显示图表
plt.grid(True)
plt.savefig("preprocessed_data/整理数据/Q1/2024-2030可行解数量.png")
plt.show()

```

Q1.4. 绘制实验次数对结果影响（python 源程序）

```

import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import shapiro, normaltest
df=pd.read_csv("preprocessed_data/整理数据/Q1/2024实验-利润.CSV")
plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像时负号 '-' 显示为方块的问题

# 将第一行作为Q2, 第二行作为Q3
df.index = ['利润']

# 转置DataFrame
df = df.T

# 计算平均值

```

```

mean_profit = df['利润'].mean()
print(f"平均值: {mean_profit}")

# 计算方差
variance_profit = df['利润'].var()
print(f"方差: {variance_profit}")

# 计算标准差
std_dev_profit = df['利润'].std()
print(f"标准差: {std_dev_profit}")

# 计算中位数
median_profit = df['利润'].median()
print(f"中位数: {median_profit}")

# 计算最大值
max_profit = df['利润'].max()
print(f"最大值: {max_profit}")

# 计算最小值
min_profit = df['利润'].min()
print(f"最小值: {min_profit}")

# 使用Shapiro-Wilk检验
stat, p = shapiro(df['利润'])
print(f'Shapiro-Wilk检验统计量: {stat}, p值: {p}')

# 判断是否符合正态分布
alpha = 0.05
if p > alpha:
    print('数据符合正态分布 (接受H0假设)')
else:
    print('数据不符合正态分布 (拒绝H0假设)')

# # 打印某一列的信息
# print(df['利润'].describe())
# 绘制折线图
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['利润'], marker='o', label='利润')
for i in range(len(df)):
    plt.annotate(f'{df["利润"][i]:.2f}', (df.index[i], df["利润"][i]), textcoords="offset
        points", xytext=(0,10), ha='center')

plt.xlabel('实验次数')
plt.ylabel('利润')
plt.legend()

# 显示图表

```

```
plt.grid(True)
plt.savefig("preprocessed_data/整理数据/Q1/2024实验-利润.png")
plt.show()
```

Q1. 作物_季度绘图 (python 源程序)

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像时负号 '-' 显示为方块的问题
# 读取数据
df = pd.read_csv('preprocessed_data/整理数据/2023年生产销售表(附件2.2).csv', encoding='gbk')
filtered_df = df[df['作物名称'].isin(['豇豆', '刀豆', '芸豆', '土豆', '西红柿', '茄子'])]

plt.rcParams.update({'font.size': 14})
# 设置作物名称为x轴，地块类型为hue，亩产量为y轴
plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df, x='作物名称', y='亩产量/斤', hue='种植季次', ci=None)
plt.savefig(f"preprocessed_data/整理数据/Q1/作物_季度_亩产量")
plt.show()

# 设置作物名称为x轴，地块类型为hue，种植成本为y轴
plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df, x='作物名称', y='种植成本/(元/亩)', hue='种植季次', ci=None)

plt.savefig(f"preprocessed_data/整理数据/Q1/作物_季度_种植成本")
plt.show()

# 设置作物名称为x轴，地块类型为hue，种植成本为y轴
plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df, x='作物名称', y='确切销售单价/(元/斤)', hue='种植季次', ci=None)
plt.ylabel('期望销售单价')
plt.savefig(f"preprocessed_data/整理数据/Q1/作物_季度_销售单价")
plt.show()
```

Q1. 作物_亩利润绘图 (python 源程序)

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像时负号 '-' 显示为方块的问题
# 读取数据
df = pd.read_csv('preprocessed_data/整理数据/2023年生产销售表(附件2.2).csv', encoding='gbk')
```



```

plot_types = df['地块类型'].unique()

for plot_type in plot_types:
    # 筛选出当前地块类型的数据
    df_plot = df[df['地块类型'] == plot_type]
    # 绘制柱形图
    plt.figure(figsize=(12, 6))
    sns.barplot(data=df_plot, x='作物名称', y='亩利润/(元/亩)', hue='种植季次', ci=None)
    plt.xlabel("作物名称", fontsize=14)
    plt.ylabel("亩利润/(元/亩)", fontsize=14)
    # 显示图表
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.savefig(f"preprocessed_data/整理数据/Q1/{plot_type}_亩利润图")
    plt.show()

```

Q1. 作物类型-亩产量、种植成本绘图（python 源程序）

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像时负号 '-' 显示为方块的问题
# 读取数据
df = pd.read_csv('preprocessed_data/整理数据/2023年生产销售表(附件2.2).csv', encoding='gbk')
# 假设你的数据框名为df
# 筛选作物名称在指定列表中的记录
filtered_df = df[df['作物名称'].isin(['黄豆', '黑豆', '红豆', '小麦', '玉米', '谷子'])]

plt.rcParams.update({'font.size': 14})
# 设置作物名称为x轴，地块类型为hue，亩产量为y轴
plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df, x='作物名称', y='亩产量/斤', hue='地块类型', ci=None)
plt.legend(fontsize='small')
plt.savefig(f"preprocessed_data/整理数据/Q1/作物1_地块_亩产量")
plt.show()

# 设置作物名称为x轴，地块类型为hue，种植成本为y轴
plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df, x='作物名称', y='种植成本/(元/亩)', hue='地块类型', ci=None)
plt.legend(fontsize='small')
plt.savefig(f"preprocessed_data/整理数据/Q1/作物1_地块_种植成本")
plt.show()

# 设置作物名称为x轴，地块类型为hue，种植成本为y轴

```

```

plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df, x='作物名称', y='确切销售单价/(元/斤)', hue='地块类型',ci=None)
plt.legend(fontsize='small')
plt.ylabel('期望销售单价') # 设置y轴标签为期望销售单价
plt.savefig(f"preprocessed_data/整理数据/Q1/作物1_地块_销售单价")
plt.show()

filtered_df1=df[df['作物名称'].isin(['豇豆','刀豆','芸豆','土豆','西红柿','茄子'])]

plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df1, x='作物名称', y='亩产量/斤', hue='地块类型',ci=None)
plt.legend(fontsize='small')
plt.savefig(f"preprocessed_data/整理数据/Q1/作物2_地块_亩产量")
plt.show()
# 设置作物名称为x轴, 地块类型为hue, 种植成本为y轴
plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df1, x='作物名称', y='种植成本/(元/亩)', hue='地块类型',ci=None)
plt.legend(fontsize='small')
plt.savefig(f"preprocessed_data/整理数据/Q1/作物2_地块_种植成本")
plt.show()

# 设置作物名称为x轴, 地块类型为hue, 种植成本为y轴
plt.figure(figsize=(12, 6))
sns.barplot(data=filtered_df1, x='作物名称', y='确切销售单价/(元/斤)', hue='地块类型',ci=None)
plt.legend(fontsize='small')
plt.ylabel('期望销售单价') # 设置y轴标签为期望销售单价
plt.savefig(f"preprocessed_data/整理数据/Q1/作物2_地块_销售单价")
plt.show()

```

Q2.1 第二问文件迁移（python 源程序）

```

import pandas as pd
yuqi2023=pd.read_csv("preprocessed_data/整理数据/Q1/2023年农作物预期销量表.csv",\
encoding='gbk')
shengchang2023=pd.read_csv("preprocessed_data/整理数据/2023年生产销售表(附件2.2).csv",\
encoding='gbk')
yuqi2023.to_csv("preprocessed_data/整理数据/Q2/2023/2023年农作物预期销量表.csv",\
encoding='gbk',index=False)
shengchang2023.to_csv("preprocessed_data/整理数据/Q2/2023/2023年生产销售表.csv",\
encoding='gbk',index=False)
ans_sheet=pd.read_csv("preprocessed_data/整理数据/Q1/2023/2023_答案表_1.csv",encoding='gbk')
ans_sheet.to_csv("preprocessed_data/整理数据/Q2/2023/2023年答案表.csv",\
encoding='gbk',index=False)

```

Q2.2 生成 2024-2030 预期销售表（python 源程序）

```
import pandas as pd
from scipy.stats import truncnorm
import numpy as np
for year in range(2024,2031):
    last_year=pd.read_csv(f"preprocessed_data/整理数据/Q2/{year-1}/{year-1}年农作物预期销量表.csv",\
encoding='gbk')
    # 定义截断正态分布的参数
    mu_default = 0
    sigma_default = 0.05 / 3
    a_default, b_default = -0.05, 0.05

    mu_6 = 0.075
    sigma_6 = 0.025 / 3
    a_6, b_6 = 0.05, 0.1
    size = last_year.shape[0]
    results = []

    for i in range(5):
        a_standard = (a_default - mu_default) / sigma_default
        b_standard = (b_default - mu_default) / sigma_default
        z = truncnorm.rvs(a_standard, b_standard, size=1)
        r = mu_default + sigma_default * z
        results.append(r)

    a_standard_6 = (a_6 - mu_6) / sigma_6
    b_standard_6 = (b_6 - mu_6) / sigma_6
    z_6 = truncnorm.rvs(a_standard_6, b_standard_6, size=1)
    r_6 = mu_6 + sigma_6 * z_6
    results.append(r_6)

    a_standard_7 = (a_6 - mu_6) / sigma_6
    b_standard_7 = (b_6 - mu_6) / sigma_6
    z_7 = truncnorm.rvs(a_standard_7, b_standard_7, size=1)
    r_7 = mu_6 + sigma_6 * z_7
    results.append(r_7)

    for i in range(6, size-1):
        a_standard = (a_default - mu_default) / sigma_default
        b_standard = (b_default - mu_default) / sigma_default
        z = truncnorm.rvs(a_standard, b_standard, size=1)
        r = mu_default + sigma_default * z
        results.append(r)

# 将结果转换为numpy数组
```

```

results = np.array(results).flatten()
last_year.iloc[:, 2] = last_year.iloc[:, 2] * (results + 1)
this_year=last_year
this_year.to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年农作物预期销量表.csv",\
encoding='gbk',index=False)

```

Q2.3 生成 2024-2030 生产销售表（python 源程序）

```

import pandas as pd
import numpy as np
from scipy.stats import truncnorm
for year in range(2024,2031):
    last_year = pd.read_csv(f"preprocessed_data/整理数据/Q2/{year - 1}/{year - 1}年生产销售表.csv",
                           encoding='gbk')

    #亩产量处理
    # 定义截断正态分布的参数
    mu = 0
    sigma = 0.1 / 3
    a, b = -0.1, 0.1
    # 标准化截断区间
    a_standard = (a - mu) / sigma
    b_standard = (b - mu) / sigma
    # 生成标准正态分布的随机数
    size = len(last_year) # 生成随机数的数量与 DataFrame 的行数相同
    z = truncnorm.rvs(a_standard, b_standard, size=size)
    # 反标准化
    r = mu + sigma * z
    # 将 df 中的亩产量/斤列乘以 (1 + r)
    last_year['亩产量/斤'] = last_year['亩产量/斤'] * (1 + r)

    #种植成本处理
    last_year['种植成本/(元/亩)']=last_year['种植成本/(元/亩)']*(1+0.05)
    r_list = np.zeros(len(last_year))
    # 对于作物编号在[1,16]区间内, 不变
    # 对于作物编号在[17,37]区间内, 增长5%
    r_list[(last_year['作物编号'] >= 17) & (last_year['作物编号'] <= 37)] = 0.05
    # 对于作物编号在[38,40]区间内, 使用截断正态分布生成随机数 r
    mu2 = -0.03
    sigma2 = 0.02 / 3
    a2, b2 = -0.05, -0.01
    # 标准化截断区间
    a2_standard = (a2 - mu2) / sigma2
    b2_standard = (b2 - mu2) / sigma2
    # 生成标准正态分布的随机数

```

```

size = len(last_year[(last_year['作物编号'] >= 38) & (last_year['作物编号'] <= 40)]) #
    生成随机数的数量
z2 = truncnorm.rvs(a2_standard, b2_standard, size=size)
# 反标准化
r2 = mu2 + sigma2 * z2
# 将 r_list 中的对应位置设置为生成的 r 值
r_list[(last_year['作物编号'] >= 38) & (last_year['作物编号'] <= 40)] = r2
# 对于作物编号在[41,41]区间内, 减少5%
r_list[(last_year['作物编号'] == 41)] = -0.05
# 将 df 中的确切销售单价/(元/斤) 列乘以 (1 + r_list)
last_year['确切销售单价/(元/斤)'] = last_year['确切销售单价/(元/斤)'] * (1 + r_list)
this_year=last_year
this_year.to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年生产销售表.csv",
    encoding='gbk', index=False)

```

Q2.4 第二问生成蒙特卡洛所需文件（python 源程序）

```

import pandas as pd

for year in range(2024,2031):
    ans_total_sheet=pd.read_csv\
        ("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
    ans_total_sheet.to_csv\
        (f"preprocessed_data/整理数据/Q2/{year}/{year}年答案表.csv",encoding='gbk',index=False)
    shengchangqingkuang=pd.read_csv\
        (f"preprocessed_data/整理数据/Q2/{year}/{year}年生产销售表.csv",encoding='gbk')
    yuqixiaoshou=pd.read_csv\
        (f"preprocessed_data/整理数据/Q2/{year}/{year}年农作物预期销量表.csv",encoding='gbk')

    #销量表
    yuqixiaoshou['种植季次'] =yuqixiaoshou['种植季次'].replace('单季', '第一季')
    pivot_df = yuqixiaoshou.pivot_table(index='种植季次', columns='作物名称', values='产量',
        aggfunc='sum')
    # 重置列索引
    pivot_df.reset_index(inplace=True)
    pivot_df.fillna(0,inplace=True)
    crop_order = yuqixiaoshou[['作物名称',
        '作物编号']].drop_duplicates().sort_values('作物编号')['作物名称'].tolist()
    pivot_df = pivot_df[['种植季次'] + crop_order]
    pivot_df.to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年_销量表.csv",\
        encoding='gbk',index=False)

    #单价表
    shengchangqingkuang['种植季次']=shengchangqingkuang['种植季次'].replace('单季','第一季')
    shengchangqingkuang['种植季次'] = shengchangqingkuang['种植季次'].replace('单季', '第一季')

```

```

df_unique = shengchangqingkuang.drop_duplicates(subset=['种植季次', '作物名称'])
danjia_df = df_unique.pivot(index='种植季次', columns='作物名称',
                             values='确切销售单价/(元/斤)')
# 按照作物编号排序列
danjia_df = danjia_df.reindex(
    sorted(danjia_df.columns, key=lambda x:
           shengchangqingkuang[shengchangqingkuang['作物名称'] == x]['作物编号'].values[0]),
    axis=1)
danjia_df.fillna(0,inplace=True)
danjia_df.to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年_单价表.csv",
                 encoding='gbk', index=False)

#成本表
df2=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
df2.fillna(0,inplace=True)
df1=pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年生产销售表.csv",encoding='gbk')
df1['种植季次']=df1['种植季次'].replace('单季', '第一季')
df1.rename(columns={'种植季次': '季度'}, inplace=True)
# 遍历 df2 的每一行
for index, row in df2.iterrows():
    matching_records = df1[(df1['地块类型'] == row['地块类型']) & (df1['季度'] ==
                                                                    row['季度'])]

    for _, record in matching_records.iterrows():
        crop_name = record['作物名称']
        planting_cost = record['种植成本/(元/亩)']

        if crop_name in df2.columns:
            df2.at[index, crop_name] = planting_cost

df2.to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年_成本表.csv", encoding='gbk',
           index=False)

#产量表
df2=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
df2.fillna(0,inplace=True)
df1=pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年生产销售表.csv",encoding='gbk')
df1['种植季次']=df1['种植季次'].replace('单季', '第一季')
df1.rename(columns={'种植季次': '季度'}, inplace=True)
for index, row in df2.iterrows():
    matching_records = df1[(df1['地块类型'] == row['地块类型']) & (df1['季度'] ==
                                                                    row['季度'])]

    for _, record in matching_records.iterrows():
        crop_name = record['作物名称']

```

```

mu_chanliang = record['亩产量/斤']

if crop_name in df2.columns:
    df2.at[index, crop_name] = mu_chanliang

# 保存更新后的 df2
df2.to_csv(f'preprocessed_data/整理数据/Q2/{year}/{year}年_产量表.csv', encoding='gbk',
           index=False)

```

Q2.5 第二问蒙特卡洛（python 源程序）

```

import random
import pandas as pd
import numpy as np
bean_index_list = [1, 2, 3, 4, 5, 17, 18, 19]
shuidao_index = 16
shuijiaodi_index = ['D1', 'D2', 'D3', 'D4', 'D5', 'D6', 'D7', 'D8']
LH_index = ['E1', 'E2', 'E3', 'E4', 'E5', 'E6', 'E7', 'E8', 'E9', 'E10', 'E11', 'E12', 'E13',
            'E14', 'E15', 'E16', 'F1', 'F2', 'F3', 'F4']
max_iter=10000
random.seed(42)
np.random.seed(42)

for year in range(2024, 2031):
    # 得到没有种植豆类年数
    year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年答案表.csv",
                           encoding='gbk')
    last_year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q2/{year-1}/{year-1}年答案表.csv",
                                encoding='gbk')
    year_ans.iloc[:, 1:42] = 0
    for index, row in year_ans.iterrows():
        # 获取对应的 last_year_ans 行
        last_year_row = last_year_ans[
            (last_year_ans['地块名'] == row['地块名']) & (last_year_ans['季度'] == row['季度'])]
        if not last_year_row.empty:
            last_year_row = last_year_row.iloc[0]
            non_bean_columns = [col for col in range(1, 42) if col not in bean_index_list]
            if (last_year_row[non_bean_columns] == 0).all():
                year_ans.at[index, '没有种植豆类年数'] = 0
            else:
                year_ans.at[index, '没有种植豆类年数'] = last_year_row['没有种植豆类年数'] + 1

    year_ans.to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年答案表.csv", encoding='gbk',
                   index=False)
    max_profit=0
    pass_num=0

```

```

print(year_ans.iloc[50,44])

def generate():
    random.seed(42)
    x = np.zeros((82,41))
    #遍历前25行
    for i in range(0,26):
        # 随机选择第2到第16列中的一个列索引
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(1, 6)
        else:
            col_index = np.random.randint(1, 16)
        # 将year_ans第i行的'地块面积/亩'的数值赋值给x的相应位置
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(26, 34):
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(17, 20)
        else:
            col_index = np.random.randint(16, 35)
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(34, 54):
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(17, 20)
            x[i, col_index - 1] = year_ans.iloc[i, 44]
        else:
            col_indices = np.random.choice(range(17, 35), 2, replace=False)
            if np.random.rand() > 0.5:
                # 第一种情况：两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
                x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
                x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
            else:
                # 第二种情况：一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
                x[i, col_indices[0]-1] = year_ans.iloc[i,44]
                x[i, col_indices[1]-1] = 0
    for i in range(54, 62):
        if x[i - 28, 16] != 0:
            x[i, :] = 0
        else:
            col_index = np.random.randint(35, 38)
            x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(62, 78):
        col_indices = np.random.choice(range(38, 42), 2, replace=False)
        if np.random.rand() > 0.5:
            # 第一种情况：两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
            x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
        else:

```



```

        # 第二种情况：一个列索引为year_ans[i]['地块面积/亩']，另一个为0
        x[i, col_indices[0]-1] = year_ans.iloc[i,44]
        x[i, col_indices[1]-1] = 0
    for i in range(78, 82):
        col_indices = np.random.choice(range(20, 35), 2, replace=False)
        if np.random.rand() > 0.5:
            # 第一种情况：两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
            x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
        else:
            # 第二种情况：一个列索引为year_ans[i]['地块面积/亩']，另一个为0
            x[i, col_indices[0]-1] = year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0
    return x

def evaluate(x):
    if (constraint4(x)):
        return True
    else:
        return False

def objective(x):
    # 将 x 重新整形为二维数组
    x = x.reshape((year_ans.shape[0], 41))
    xiaoliang = pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年_销量表.csv",
        encoding='gbk').iloc[:,
        1:].values
    danjia = pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年_单价表.csv",
        encoding='gbk').values
    chengben = pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年_成本表.csv",
        encoding='gbk').iloc[:,
        1:42].values
    chanliang=pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年_产量表.csv",
        encoding='gbk').iloc[:,1:42].values
    # 确保 danjia 和 chengben 中的所有数据都是数值型
    danjia = danjia.astype(float)
    chengben = chengben.astype(float)
    mu_x=x.copy()

    x=x*chanliang
    # 计算 x 的列和
    x_sum_1 = np.sum(x[0:54, :], axis=0)
    x_sum_2 = np.sum(x[54:, :], axis=0)

    # 对每一列进行 min 操作并计算销售金额
    x_min_1 = np.minimum(x_sum_1, xiaoliang[0, :])

```

```

x_min_2 = np.minimum(x_sum_2, xiaoliang[1, :])

xiaoshou_jine = np.sum(x_min_1 * danjia[0, :]) + np.sum(x_min_2 * danjia[1, :])

# 计算成本
chengben_jine = np.multiply(mu_x, chengben)

# 计算利润
profit = xiaoshou_jine - np.sum(chengben_jine)

return profit

def constraint4(x):
    x = x.reshape((year_ans.shape[0], 41))
    constraints = []
    yijidu_list = [i for i in range(26, 54)]
    for i in yijidu_list:
        for j in range(1, 42):
            aij = float(last_year_ans.iloc[i + 28, j])

            bij = float(year_ans.iloc[i + 28, j])
            if ((x[i, j - 1] * (aij + bij)) != 0):
                return False
    return True

for i in range(max_iter):
    x=generate()
    if(evaluate(x)):
        pass_num=pass_num+1
        new_profit=objective(x)
        if new_profit>max_profit:
            print(year, ":", "new_profit", new_profit)
            pd.DataFrame(x).to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年解.csv", \
                encoding='gbk', index=False)
            max_profit=new_profit

with open(f"preprocessed_data/整理数据/Q2/{year}/{year}年利润.txt", 'w', encoding='utf-8')
    as file:
        file.write(str(max_profit))
        file.write('\n')
        file.write(str(pass_num))

```

```

jie=pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年解.csv",encoding='gbk')
ans=pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年答案表.csv",encoding='gbk')
# 打印形状以进行调试
ans.iloc[:, 1:42] = jie.values
ans.to_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年答案表.csv",encoding='gbk',\
index=False)

```

Q3.1 第三问文件迁移（python 源程序）

```

import pandas as pd

yuqi_2023=pd.read_csv("preprocessed_data/整理数据/Q2/2023/2023年农作物预期销量表.csv",\
encoding='gbk')
shengchan2023=pd.read_csv("preprocessed_data/整理数据/Q2/2023/2023年生产销售表.csv",\
encoding='gbk')
yuqi_2023.to_csv("preprocessed_data/整理数据/Q3/2023/2023年农作物预期销量表.csv",\
encoding='gbk',index=False)
shengchan2023.to_csv("preprocessed_data/整理数据/Q3/2023/2023年生产销售表.csv",\
encoding='gbk',index=False)
ans_sheet=pd.read_csv("preprocessed_data/整理数据/Q1/2023/2023_答案表_1.csv",\
encoding='gbk')
ans_sheet.to_csv(f"preprocessed_data/整理数据/Q3/2023/2023年答案表.csv",\
encoding='gbk',index=False)
for year in range(2024,2031):
    shengchan=pd.read_csv(f"preprocessed_data/整理数据/Q2/{year}/{year}年生产销售表.csv",\
encoding='gbk')
    shengchan.to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年生产销售表.csv",\
encoding='gbk',index=False)

```

Q3.2 生成新 2024-2030 预期产量表（python 源程序）

```

import pandas as pd
from scipy.stats import truncnorm
import numpy as np
for year in range(2024,2031):
    last_year=pd.read_csv\
(f"preprocessed_data/整理数据/Q3/{year-1}/{year-1}年农作物预期销量表.csv",encoding='gbk')
    # 定义截断正态分布的参数
    mu_default = 0
    sigma_default = 0.05 / 3
    a_default, b_default = -0.05, 0.05

    mu_6 = 0.075
    sigma_6 = 0.025 / 3
    a_6, b_6 = 0.05, 0.1

```

```

size = last_year.shape[0]
results = []

for i in range(5):
    a_standard = (a_default - mu_default) / sigma_default
    b_standard = (b_default - mu_default) / sigma_default
    z = truncnorm.rvs(a_standard, b_standard, size=1)
    r = mu_default + sigma_default * z
    results.append(r)

a_standard_6 = (a_6 - mu_6) / sigma_6
b_standard_6 = (b_6 - mu_6) / sigma_6
z_6 = truncnorm.rvs(a_standard_6, b_standard_6, size=1)
r_6 = mu_6 + sigma_6 * z_6
results.append(r_6)

a_standard_7 = (a_6 - mu_6) / sigma_6
b_standard_7 = (b_6 - mu_6) / sigma_6
z_7 = truncnorm.rvs(a_standard_7, b_standard_7, size=1)
r_7 = mu_6 + sigma_6 * z_7
results.append(r_7)

for i in range(6, size-1):
    a_standard = (a_default - mu_default) / sigma_default
    b_standard = (b_default - mu_default) / sigma_default
    z = truncnorm.rvs(a_standard, b_standard, size=1)
    r = mu_default + sigma_default * z
    results.append(r)

# 将结果转换为numpy数组
results = np.array(results).flatten()
#同类农作物影响
# [1-16] 增长率为0, [17,37] 增长率为0.05*a(a=0.1),
# [38,40] 增长率为(-0.03*a(a=0.1)), [41], 增长率为-0.05*a(a=0.1)
a = 0.1
r2 = np.zeros(size)
crop_numbers = last_year['作物编号']

r2[(crop_numbers >= 1) & (crop_numbers <= 16)] = 0
r2[(crop_numbers >= 17) & (crop_numbers <= 37)] = 0.05 * a
r2[(crop_numbers >= 38) & (crop_numbers <= 40)] = -0.03 * a
r2[(crop_numbers == 41)] = -0.05 * a

#灰色关联分析
# [1-16] 增长率为(1-0.748*0.05), [17,37] 增长率为(1-0.807*0.05)*(1-0.748*0.05),
# [38,40] 增长率为(1-0.807*(-0.03))*(1-0.748*0.05), [41], 增长率为(1-0.807*(-0.05))*(1-0.748*0.05)

```

```

r3 = np.zeros(size)
r3[(crop_numbers >= 1) & (crop_numbers <= 16)] = -(0.849 * 0.05)
r3[(crop_numbers >= 17) & (crop_numbers <= 37)] = -((0.864 * 0.05) + (0.849 * 0.05))
r3[(crop_numbers >= 38) & (crop_numbers <= 40)] = -((0.864 * (-0.03)) + (0.849 * 0.05))
r3[(crop_numbers == 41)] = -((0.864 * (-0.05)) + (0.849 * 0.05))

# 合并 results, r2 和 r3
combined_results = results + r2 + r3

last_year.iloc[:, 2] = last_year.iloc[:, 2] * (combined_results + 1)
this_year=last_year
this_year.to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年农作物预期销量表.csv",\
encoding='gbk',index=False)

```

Q3.3 第三问生成蒙特卡洛所需文件（python 源程序）

```

import pandas as pd

for year in range(2024,2031):
    ans_total_sheet=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
    ans_total_sheet.to_csv\
(f"preprocessed_data/整理数据/Q3/{year}/{year}年答案表.csv",encoding='gbk',index=False)
    shengchangqingkuang=pd.read_csv\
(f"preprocessed_data/整理数据/Q3/{year}/{year}年生产销售表.csv",encoding='gbk')
    yuqixiaoshou=pd.read_csv\
(f"preprocessed_data/整理数据/Q3/{year}/{year}年农作物预期销量表.csv",encoding='gbk')

    #销量表
    yuqixiaoshou['种植季次'] =yuqixiaoshou['种植季次'].replace('单季', '第一季')
    pivot_df = yuqixiaoshou.pivot_table(index='种植季次', columns='作物名称', values='产量',
        aggfunc='sum')
    # 重置列索引
    pivot_df.reset_index(inplace=True)
    pivot_df.fillna(0,inplace=True)
    crop_order = yuqixiaoshou[['作物名称',
        '作物编号']].drop_duplicates().sort_values('作物编号')['作物名称'].tolist()
    pivot_df = pivot_df[['种植季次'] + crop_order]
    pivot_df.to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年_销量表.csv",\
encoding='gbk',index=False)

    #单价表
    shengchangqingkuang['种植季次']=shengchangqingkuang['种植季次'].replace('单季','第一季')
    shengchangqingkuang['种植季次'] = shengchangqingkuang['种植季次'].replace('单季', '第一季')
    df_unique = shengchangqingkuang.drop_duplicates(subset=['种植季次', '作物名称'])
    danjia_df = df_unique.pivot(index='种植季次', columns='作物名称',

```

```

        values='确切销售单价/(元/斤)')
danjia_df = danjia_df.reindex(
    sorted(danjia_df.columns, key=lambda x:
        shengchangqingkuang[shengchangqingkuang['作物名称'] == x]['作物编号'].values[0]),
        axis=1)
danjia_df.fillna(0,inplace=True)
danjia_df.to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年_单价表.csv",
    encoding='gbk', index=False)

#成本表
df2=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
df2.fillna(0,inplace=True)
df1=pd.read_csv(f'preprocessed_data/整理数据/Q3/{year}/{year}年生产销售表.csv',encoding='gbk')
df1['种植季次']=df1['种植季次'].replace('单季', '第一季')
df1.rename(columns={'种植季次': '季度'}, inplace=True)
for index, row in df2.iterrows():
    matching_records = df1[(df1['地块类型'] == row['地块类型']) & (df1['季度'] ==
        row['季度'])]

    for _, record in matching_records.iterrows():
        crop_name = record['作物名称']
        planting_cost = record['种植成本/(元/亩)']

        # 更新 df2 中对应列的数据
        if crop_name in df2.columns:
            df2.at[index, crop_name] = planting_cost

# 保存更新后的 df2
df2.to_csv(f'preprocessed_data/整理数据/Q3/{year}/{year}年_成本表.csv', encoding='gbk',
    index=False)

#产量表
df2=pd.read_csv("preprocessed_data/整理数据/答案总表模板.csv",encoding='gbk')
df2.fillna(0,inplace=True)
df1=pd.read_csv(f'preprocessed_data/整理数据/Q3/{year}/{year}年生产销售表.csv',encoding='gbk')
df1['种植季次']=df1['种植季次'].replace('单季', '第一季')
df1.rename(columns={'种植季次': '季度'}, inplace=True)
for index, row in df2.iterrows():
    matching_records = df1[(df1['地块类型'] == row['地块类型']) & (df1['季度'] ==
        row['季度'])]

    for _, record in matching_records.iterrows():
        crop_name = record['作物名称']
        mu_chanliang = record['亩产量/斤']

        # 更新 df2 中对应列的数据

```

```

        if crop_name in df2.columns:
            df2.at[index, crop_name] = mu_chanliang

# 保存更新后的 df2
df2.to_csv(f'preprocessed_data/整理数据/Q3/{year}/{year}年产量表.csv', encoding='gbk',
            index=False)

```

Q3.4 第三问蒙特卡洛（python 源程序）

```

import random
import pandas as pd
import numpy as np
bean_index_list = [1, 2, 3, 4, 5, 17, 18, 19]
shuidao_index = 16
shuijiaodi_index = ['D1', 'D2', 'D3', 'D4', 'D5', 'D6', 'D7', 'D8']
LH_index = ['E1', 'E2', 'E3', 'E4', 'E5', 'E6', 'E7', 'E8', 'E9', 'E10', 'E11', 'E12', 'E13',
            'E14', 'E15', 'E16', 'F1', 'F2', 'F3', 'F4']
max_iter=10000
random.seed(8)
np.random.seed(8)

for year in range(2024, 2031):
    # 得到没有种植豆类年数
    year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年答案表.csv",
                           encoding='gbk')
    last_year_ans = pd.read_csv(f"preprocessed_data/整理数据/Q3/{year-1}/{year-1}年答案表.csv",
                                encoding='gbk')
    year_ans.iloc[:, 1:42] = 0
    for index, row in year_ans.iterrows():
        # 获取对应的 last_year_ans 行
        last_year_row = last_year_ans[
            (last_year_ans['地块名'] == row['地块名']) & (last_year_ans['季度'] == row['季度'])]
        if not last_year_row.empty:
            last_year_row = last_year_row.iloc[0]
            non_bean_columns = [col for col in range(1, 42) if col not in bean_index_list]
            if (last_year_row[non_bean_columns] == 0).all():
                year_ans.at[index, '没有种植豆类年数'] = 0
            else:
                year_ans.at[index, '没有种植豆类年数'] = last_year_row['没有种植豆类年数'] + 1

    year_ans.to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年答案表.csv", encoding='gbk',
                    index=False)
    max_profit=0
    pass_num=0

```

```

def generate():
    random.seed(8)
    x = np.zeros((82,41))
    #遍历前25行
    for i in range(0,26):
        # 随机选择第2到第16列中的一个列索引
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(1, 6)
        else:
            col_index = np.random.randint(1, 16)
        # 将year_ans第i行的'地块面积/亩'的数值赋值给x的相应位置
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(26, 34):
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(17, 20)
        else:
            col_index = np.random.randint(16, 35)
        x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(34, 54):
        if year_ans.iloc[i,45]>=2:
            col_index=np.random.randint(17, 20)
            x[i, col_index - 1] = year_ans.iloc[i, 44]
        else:
            col_indices = np.random.choice(range(17, 35), 2, replace=False)
            if np.random.rand() > 0.5:
                # 第一种情况：两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
                x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
                x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
            else:
                # 第二种情况：一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
                x[i, col_indices[0]-1] = year_ans.iloc[i,44]
                x[i, col_indices[1]-1] = 0
    for i in range(54, 62):
        if x[i - 28, 16] != 0:
            x[i, :] = 0
        else:
            col_index = np.random.randint(35, 38)
            x[i, col_index-1] = year_ans.iloc[i,44]
    for i in range(62, 78):
        col_indices = np.random.choice(range(38, 42), 2, replace=False)
        if np.random.rand() > 0.5:
            # 第一种情况：两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
            x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
        else:
            # 第二种情况：一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
            x[i, col_indices[0]-1] = year_ans.iloc[i,44]

```



```

        x[i, col_indices[1]-1] = 0
    for i in range(78, 82):
        col_indices = np.random.choice(range(20, 35), 2, replace=False)
        if np.random.rand() > 0.5:
            # 第一种情况: 两个列索引的值都为0.5 * year_ans[i]['地块面积/亩']
            x[i, col_indices[0]-1] = 0.5 * year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0.5 * year_ans.iloc[i,44]
        else:
            # 第二种情况: 一个列索引为year_ans[i]['地块面积/亩'], 另一个为0
            x[i, col_indices[0]-1] = year_ans.iloc[i,44]
            x[i, col_indices[1]-1] = 0
    return x

def evaluate(x):
    if (constraint4(x)):
        return True
    else:
        return False

def objective(x):
    # 将 x 重新整形为二维数组
    random.seed(42)
    x = x.reshape((year_ans.shape[0], 41))
    xiaoliang = pd.read_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年_销量表.csv",
        encoding='gbk').iloc[:,
        1:].values
    danjia = pd.read_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年_单价表.csv",
        encoding='gbk').values
    chengben = pd.read_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年_成本表.csv",
        encoding='gbk').iloc[:,
        1:42].values
    chanliang=pd.read_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年_产量表.csv",
        encoding='gbk').iloc[:,1:42].values
    # 确保 danjia 和 chengben 中的所有数据都是数值型
    danjia = danjia.astype(float)
    chengben = chengben.astype(float)
    mu_x=x.copy()
    # x=x*chanliang
    result = np.zeros((82, 41))
    # 遍历每一行
    for i in range(0,54):
        # 获取 "没有种植豆类年数" 属性
        no_bean_years = year_ans.iloc[i]['没有种植豆类年数']
        row_result = x[i] * chanliang[i]
        if no_bean_years == 0:
            row_result *= 1.25
        # 存储结果
        result[i] = row_result

```

```

for i in range(54,82):
    row_result = x[i] * chanliang[i]
    if (year_ans.iloc[i - 28, bean_index_list] != 0).any():
        row_result *= 1.25
    result[i] = row_result
x=result
# 计算 x 的列和
x_sum_1 = np.sum(x[0:54, :], axis=0)
x_sum_2 = np.sum(x[54:, :], axis=0)

# 对每一列进行 min 操作并计算销售金额
x_min_1 = np.minimum(x_sum_1, xiaoliang[0, :])
x_min_2 = np.minimum(x_sum_2, xiaoliang[1, :])

xiaoshou_jine = np.sum(x_min_1 * danjia[0, :]) + np.sum(x_min_2 * danjia[1, :])

# 计算成本
chengben_jine = np.multiply(mu_x, chengben)
# 计算利润
profit = xiaoshou_jine - np.sum(chengben_jine)
return profit

def constraint4(x):
    x = x.reshape((year_ans.shape[0], 41))
    constraints = []
    yijidu_list = [i for i in range(26, 54)]
    for i in yijidu_list:
        for j in range(1, 42):
            aij = float(last_year_ans.iloc[i + 28, j])

            bij = float(year_ans.iloc[i + 28, j])
            if ((x[i, j - 1] * (aij + bij)) != 0):
                return False
    return True

# if year==2024:
for j in range(1,11):
    profit=0
    pass_num_1=0
    for i in range(1000):
        x = generate()
        if (evaluate(x)):
            pass_num = pass_num + 1
            pass_num_1=pass_num_1+1
            new_profit = objective(x)
            if new_profit > max_profit:

```

```

        print(year, ":", "new_profit", new_profit)
        pd.DataFrame(x).to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年解.csv",
                               encoding='gbk',
                               index=False)

        max_profit = new_profit
        if new_profit>profit:
            profit=new_profit
            pd.DataFrame(x).to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}_{j}年解.csv",
                                    encoding='gbk',
                                    index=False)

    with open(f"preprocessed_data/整理数据/Q3/{year}/{year}_{j}年利润.txt", 'w',
              encoding='utf-8') as file:
        file.write(str(profit))
        file.write('\n')
        file.write(str(pass_num_1))

with open(f"preprocessed_data/整理数据/Q3/{year}/{year}年利润.txt", 'w', encoding='utf-8')
    as file:
        file.write(str(max_profit))
        file.write('\n')
        file.write(str(pass_num))

jie=pd.read_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年解.csv",encoding='gbk')
ans=pd.read_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年答案表.csv",encoding='gbk')
# 打印形状以进行调试
ans.iloc[:, 1:42] = jie.values
ans.to_csv(f"preprocessed_data/整理数据/Q3/{year}/{year}年答案表.csv",encoding='gbk',index=False)

```

Q3.6 绘图 Q2Q3 对比（python 源程序）

```

import pandas as pd
import matplotlib.pyplot as plt
df=pd.read_csv("preprocessed_data/整理数据/Q3/Q2Q3对比.CSV")
plt.rcParams['font.sans-serif'] = ['SimHei'] # 指定默认字体为黑体
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像时负号 '-' 显示为方块的问题
print(df)

# 将第一行作为Q2，第二行作为Q3
df.index = ['Q2', 'Q3']

# 转置DataFrame
df = df.T

```

```
# 绘制折线图
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['Q2'], marker='o', label='Q2')
plt.plot(df.index, df['Q3'], marker='o', label='Q3')
for i in range(len(df)):
    plt.annotate(f'{df["Q2"][i]:.2f}', (df.index[i], df["Q2"][i]), textcoords="offset points",
                xytext=(0,10), ha='center')
    plt.annotate(f'{df["Q3"][i]:.2f}', (df.index[i], df["Q3"][i]), textcoords="offset points",
                xytext=(0,10), ha='center')

plt.xlabel('年份')
plt.ylabel('总利润')
plt.legend()

# 显示图表
plt.grid(True)
plt.savefig("preprocessed_data/整理数据/Q3/Q2Q3对比.png")
plt.show()
```